

Candidate Political Positioning Analysis

nadia

2026-05-07

```
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE, fig.width = 8, fig.height = 6)
```

```
#Load all required libraries
```

```
library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(factoextra)
library(psych)
library(caret)
library(randomForest)
library(nnet)
library(MASS)
library(fastshap)
library(tibble)
library(igraph)
library(tidygraph)
library(gggraph)
library(visNetwork)
library(viridis)
library(pheatmap)
library(ggcorrplot)
library(vip)
library(fpc)
library(stringr)
library(cluster)
```

```
#Set seed for reproducibility
set.seed(123)
```

```
#Helper functions
```

```
make_safe_levels <- function(f) {
  safe_levels <- make.names(levels(f))
  factor(make.names(as.character(f)), levels = safe_levels)
}
```

```
#Min-max normalization to 0-1 scale
```

```
normalize_01 <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}
```

```
# Data Loading and Initial Cleaning
```

```
#Load data
```

```
df <- read_csv("C:/Users/nadia/Desktop/pilot/altinget_answers_playwright_271.csv",
  show_col_types = FALSE)
```

```
#Normalize candidate name column
```

```
if (!"candidate_name" %in% names(df) & "name" %in% names(df)) {
  df <- rename(df, candidate_name = name)
}
```

```
#Detect question columns (Q1-Q26)
```

```
question_cols <- grep("^Q", names(df), value = TRUE)
cat("Detected", length(question_cols), "question columns\n")
```

```
## Detected 26 question columns
```

```
#Initial filtering: complete cases only
before_n <- nrow(df)
df <- df %>%
filter(!is.na(party) & party != "") %>%
filter(if_all(all_of(question_cols), ~ !is.na(.))) %>%
mutate(across(all_of(question_cols), as.numeric))

cat("Rows before filtering:", before_n, "\n")
```

```
## Rows before filtering: 199
```

```
cat("Rows after filtering:", nrow(df), "\n")
```

```
## Rows after filtering: 155
```

```
# Exploratory Data Analysis (Pre-Filtering)
```

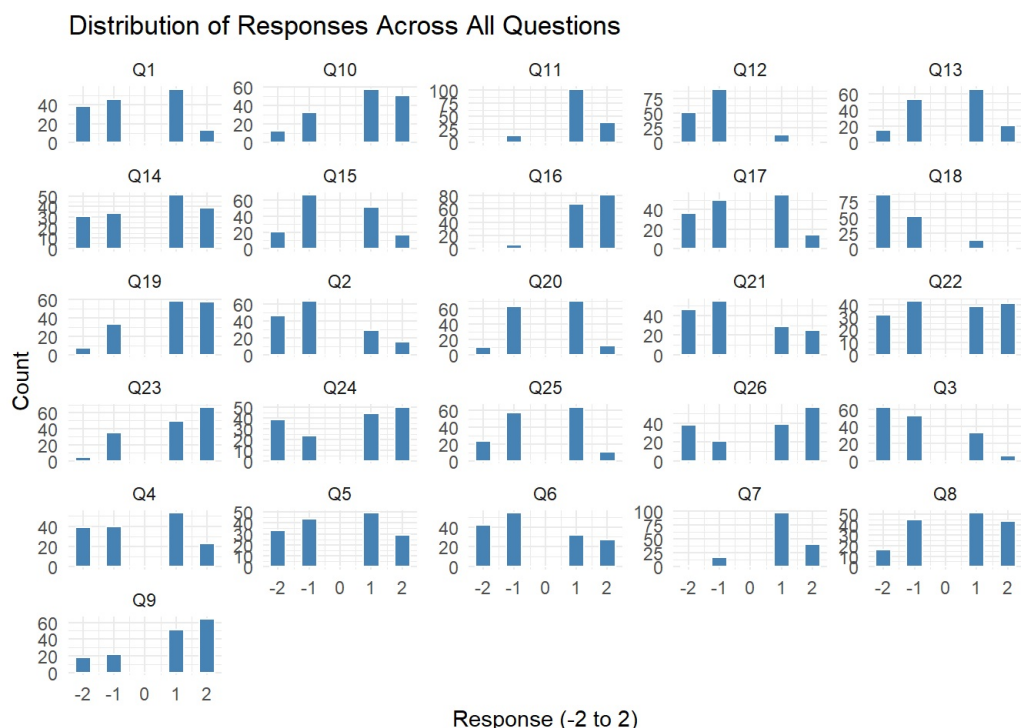
```
## Response Distributions
```

```
#Summary statistics for all questions
summary(select(df, all_of(question_cols)))
```

```
##          Q1          Q2          Q3          Q4
## Min.   :-2.0000 Min.   :-2.0000 Min.   :-2.0000 Min.   :-2.0000
## 1st Qu.: -1.5000 1st Qu.: -2.0000 1st Qu.: -2.0000 1st Qu.: -1.5000
## Median :-1.0000 Median :-1.0000 Median :-1.0000 Median :-1.0000
## Mean   :-0.2645 Mean   :-0.6387 Mean   :-0.8645 Mean   :-0.1226
## 3rd Qu.: 1.0000 3rd Qu.: 1.0000 3rd Qu.: 0.0000 3rd Qu.: 1.0000
## Max.    : 2.0000 Max.    : 2.0000 Max.    : 2.0000 Max.    : 2.0000
##          Q5          Q6          Q7          Q8
## Min.   :-2.00000 Min.   :-2.0000 Min.   :-2 Min.   :-2.0000
## 1st Qu.: -1.00000 1st Qu.: -2.0000 1st Qu.: 1 1st Qu.: -1.0000
## Median : 1.00000 Median :-1.0000 Median : 1 Median : 1.0000
## Mean   :-0.01935 Mean   :-0.3355 Mean   : 1 Mean   : 0.3871
## 3rd Qu.: 1.00000 3rd Qu.: 1.0000 3rd Qu.: 2 3rd Qu.: 2.0000
## Max.    : 2.00000 Max.    : 2.0000 Max.    : 2 Max.    : 2.0000
##          Q9          Q10         Q11         Q12
## Min.   :-2.0000 Min.   :-2.0000 Min.   :-2.000 Min.   :-2.000
## 1st Qu.: -1.0000 1st Qu.: -1.0000 1st Qu.: 1.000 1st Qu.: -2.000
## Median : 1.0000 Median : 1.0000 Median : 1.000 Median :-1.000
## Mean    : 0.7806 Mean    : 0.6516 Mean    : 1.039 Mean    :-1.161
## 3rd Qu.: 2.0000 3rd Qu.: 2.0000 3rd Qu.: 1.000 3rd Qu.: -1.000
## Max.    : 2.0000 Max.    : 2.0000 Max.    : 2.000 Max.    : 1.000
##          Q13         Q14         Q15         Q16
## Min.   :-2.0000 Min.   :-2.0000 Min.   :-2.0000 Min.   :-1.000
## 1st Qu.: -1.0000 1st Qu.: -1.0000 1st Qu.: -1.0000 1st Qu.: 1.000
## Median : 1.0000 Median : 1.0000 Median :-1.0000 Median : 2.000
## Mean    : 0.1613 Mean    : 0.2129 Mean    :-0.1484 Mean    : 1.445
## 3rd Qu.: 1.0000 3rd Qu.: 1.5000 3rd Qu.: 1.0000 3rd Qu.: 2.000
## Max.    : 2.0000 Max.    : 2.0000 Max.    : 2.0000 Max.    : 2.000
##          Q17         Q18         Q19         Q20
## Min.   :-2.0000 Min.   :-2.000 Min.   :-2.0000 Min.   :-2.00000
## 1st Qu.: -1.0000 1st Qu.: -2.000 1st Qu.: -1.0000 1st Qu.: -1.00000
## Median :-1.0000 Median :-2.000 Median : 1.0000 Median : 1.00000
## Mean   :-0.2516 Mean   :-1.316 Mean    : 0.8065 Mean    : 0.07097
## 3rd Qu.: 1.0000 3rd Qu.: -1.000 3rd Qu.: 2.0000 3rd Qu.: 1.00000
## Max.    : 2.0000 Max.    : 2.000 Max.    : 2.0000 Max.    : 2.00000
##          Q21         Q22         Q23         Q24
## Min.   :-2.0000 Min.   :-2.00000 Min.   :-2.0000 Min.   :-2.0000
## 1st Qu.: -2.0000 1st Qu.: -1.00000 1st Qu.: 0.0000 1st Qu.: -1.0000
## Median :-1.0000 Median : 1.00000 Median : 1.0000 Median : 1.0000
## Mean   :-0.4387 Mean    : 0.09032 Mean    : 0.9032 Mean    : 0.2903
## 3rd Qu.: 1.0000 3rd Qu.: 2.00000 3rd Qu.: 2.0000 3rd Qu.: 2.0000
## Max.    : 2.0000 Max.    : 2.00000 Max.    : 2.0000 Max.    : 2.0000
##          Q25         Q26
## Min.   :-2.0000 Min.   :-2.0000
## 1st Qu.: -1.0000 1st Qu.: -1.0000
## Median :-1.0000 Median : 1.0000
## Mean   :-0.1097 Mean    : 0.3613
## 3rd Qu.: 1.0000 3rd Qu.: 2.0000
## Max.    : 2.0000 Max.    : 2.0000
```

```
#Visualize response distributions
df_long <- df %>%
pivot_longer(all_of(question_cols), names_to = "Question", values_to = "Value")

ggplot(df_long, aes(x = Value)) +
geom_histogram(binwidth = 0.5, color = "white", fill = "steelblue") +
facet_wrap(~Question, scales = "free_y", ncol = 5) +
theme_minimal() +
labs(title = "Distribution of Responses Across All Questions",
x = "Response (-2 to 2)", y = "Count")
```



```
## Party Distribution (Before Filtering)
```

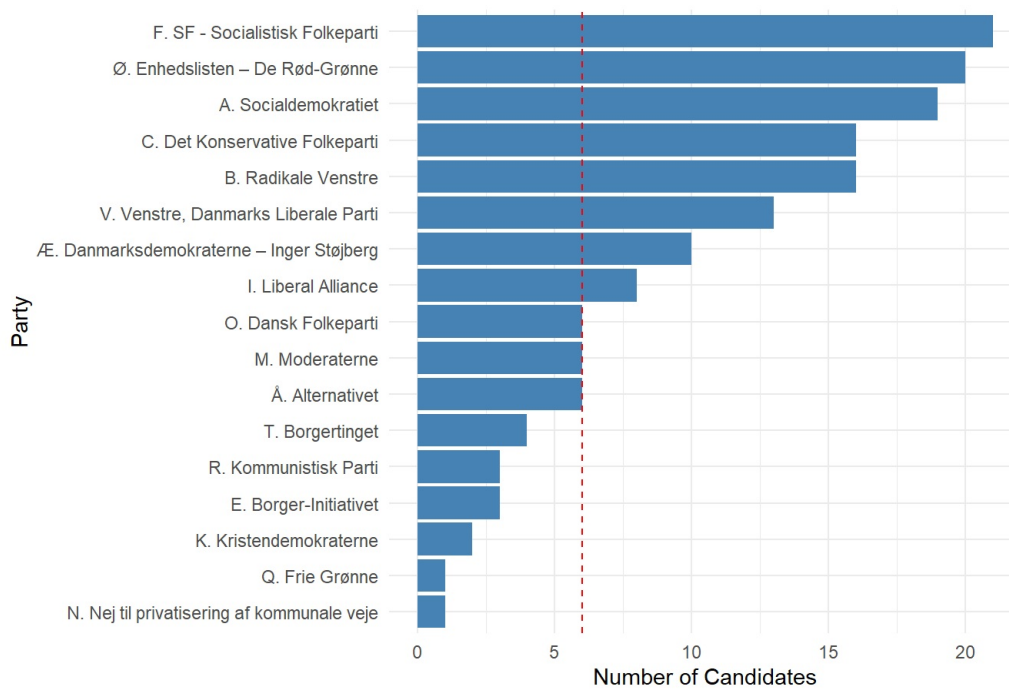
```
#Count candidates per party
party_counts <- df %>%
count(party, name = "n") %>%
arrange(desc(n))

print(party_counts)
```

```
## # A tibble: 17 × 2
##   party                                n
##   <chr>                                <int>
## 1 F. SF - Socialistisk Folkeparti      21
## 2 Ø. Enhedslisten – De Rød-Grønne     20
## 3 A. Socialdemokratiet                 19
## 4 B. Radikale Venstre                  16
## 5 C. Det Konservative Folkeparti       16
## 6 V. Venstre, Danmarks Liberale Parti   13
## 7 Æ. Danmarksdemokraterne – Inger Støjberg 10
## 8 I. Liberal Alliance                   8
## 9 M. Moderaterne                       6
## 10 O. Dansk Folkeparti                  6
## 11 Å. Alternativet                      6
## 12 T. Borgertinget                     4
## 13 E. Borger-Initiativet                 3
## 14 R. Kommunistisk Parti                 3
## 15 K. Kristendemokraterne                2
## 16 N. Nej til privatisering af kommunale veje 1
## 17 Q. Frie Grønne                       1
```

```
ggplot(party_counts, aes(x = reorder(party, n), y = n)) +
geom_col(fill = "steelblue") +
geom_hline(yintercept = 6, linetype = "dashed", color = "red") +
coord_flip() +
theme_minimal() +
labs(title = "Candidates per Party (Red line = threshold of 6)",
x = "Party", y = "Number of Candidates")
```

Candidates per Party (Red line = threshold of 6)



Mean and Median Responses by Party (Pre-Filtering)

```
#Calculate mean and median for each party-question combination
```

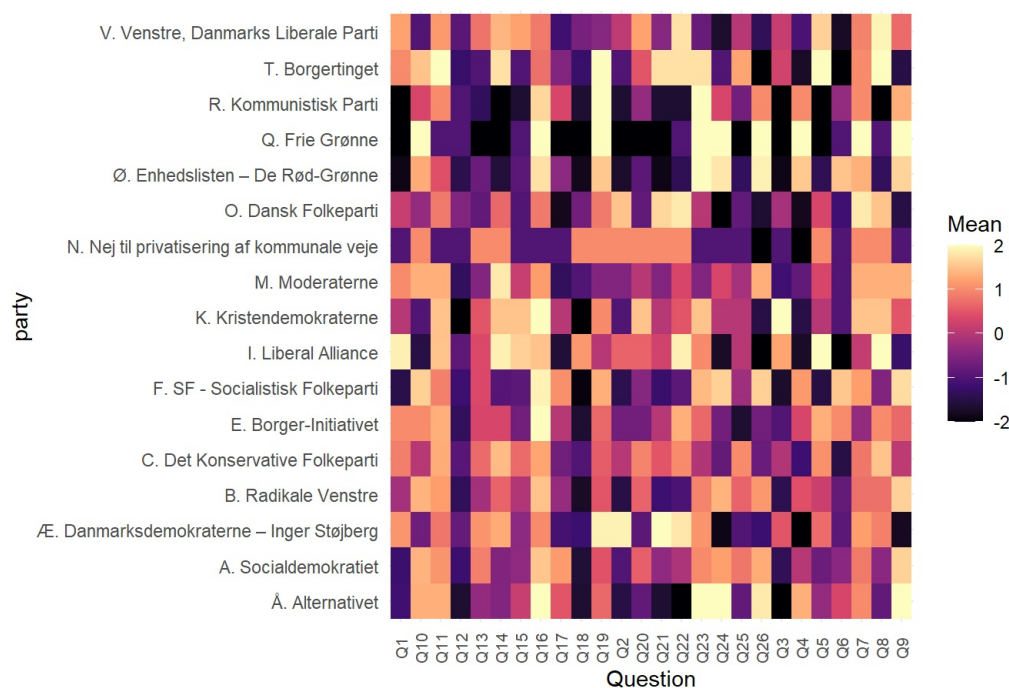
```
party_summary <- df %>%
  group_by(party) %>%
  summarise(across(all_of(question_cols),
    list(mean = ~mean(., na.rm=TRUE),
          median = ~median(., na.rm=TRUE))),
    .groups = "drop")
```

```
#Create mean matrix for heatmap
```

```
mean_long <- party_summary %>%
  select(party, ends_with("_mean")) %>%
  pivot_longer(-party, names_to = "Question", values_to = "Mean") %>%
  mutate(Question = str_remove(Question, " mean"))
```

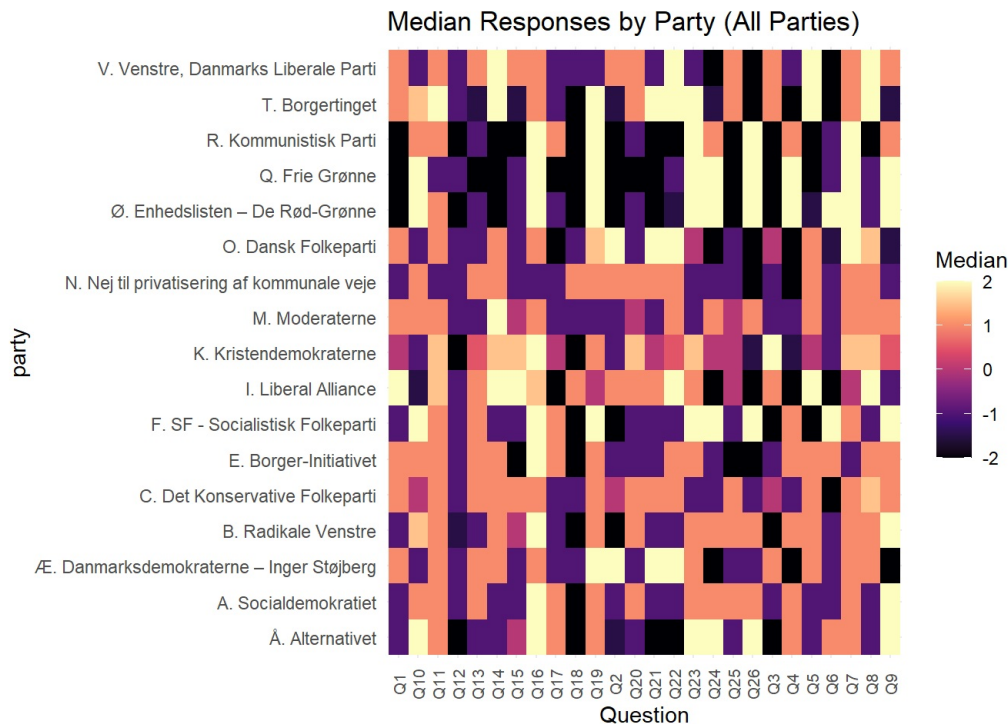
```
ggplot(mean_long, aes(x = Question, y = party, fill = Mean)) +
  geom_tile() +
  scale_fill_viridis_c(option = "magma") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, size = 8)) +
  labs(title = "Mean Responses by Party (All Parties)")
```

Mean Responses by Party (All Parties)



```
#Create median matrix for heatmap
median_long <- party_summary %>%
  select(party, ends_with("_median")) %>%
  pivot_longer(-party, names_to = "Question", values_to = "Median") %>%
  mutate(Question = str_remove(Question, "_median"))

ggplot(median_long, aes(x = Question, y = party, fill = Median)) +
  geom_tile() +
  scale_fill_viridis_c(option = "magma") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, size = 8)) +
  labs(title = "Median Responses by Party (All Parties)")
```



Question Variance Analysis

```
#Compute variance and SD per question
variance_df <- df %>%
  select(all_of(question_cols)) %>%
  summarise(across(everything(), var, na.rm = TRUE)) %>%
  pivot_longer(cols = everything(), names_to = "Question", values_to = "Variance")

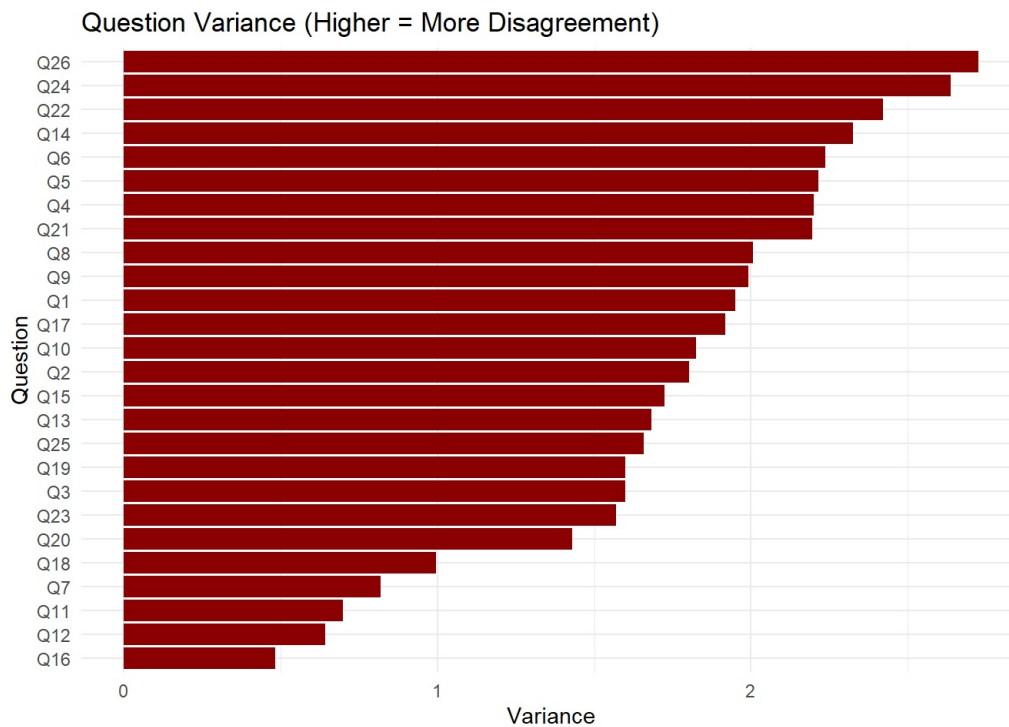
sd_df <- df %>%
  select(all_of(question_cols)) %>%
  summarise(across(everything(), sd, na.rm = TRUE)) %>%
  pivot_longer(cols = everything(), names_to = "Question", values_to = "SD")

variance_summary <- variance_df %>%
  left_join(sd_df, by = "Question") %>%
  arrange(desc(Variance))

print(variance_summary)
```

```
## # A tibble: 26 × 3
##   Question Variance    SD
##   <chr>      <dbl> <dbl>
## 1 Q26       2.73  1.65
## 2 Q24       2.64  1.62
## 3 Q22       2.42  1.56
## 4 Q14       2.32  1.52
## 5 Q6        2.24  1.50
## 6 Q5        2.21  1.49
## 7 Q4        2.20  1.48
## 8 Q21       2.20  1.48
## 9 Q8        2.01  1.42
## 10 Q9       1.99  1.41
## # i 16 more rows
```

```
ggplot(variance_summary, aes(x = reorder(Question, Variance), y = Variance)) +
  geom_col(fill = "darkred") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Question Variance (Higher = More Disagreement)",
       x = "Question", y = "Variance")
```

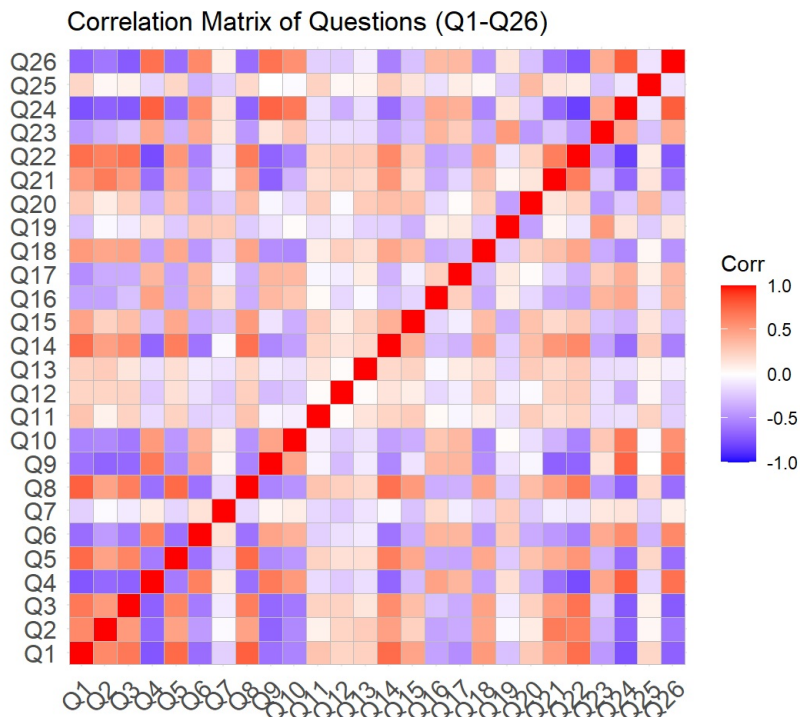


```
# Internal Consistency and Item Analysis
```

```
#Correlation matrix of questions
```

```
corr_matrix <- cor(select(df, all_of(question_cols)), use = "pairwise.complete.obs")
```

```
ggcorrplot(corr_matrix, lab = FALSE,
           title = "Correlation Matrix of Questions (Q1-Q26)")
```



```
#Cronbach's alpha with automatic key checking
```

```
alpha_initial <- psych::alpha(select(df, all_of(question_cols)),
                              check.keys = TRUE)
```

```
cat("\n=== CRONBACH'S ALPHA (BEFORE REVERSAL) ===\n")
```

```
##  
## === CRONBACH'S ALPHA (BEFORE REVERSAL) ===
```

```
print(alpha_initial$total)
```

```
## raw_alpha std.alpha G6(smc) average_r S/N ase mean  
## 0.9357037 0.9303083 0.9550076 0.3392446 13.3489 0.006625487 -0.3280397  
## sd median_r  
## 0.8178246 0.3182825
```

```
#Extract item statistics  
item_stats <- alpha_initial$item.stats %>%  
rownames_to_column("Question") %>%  
as_tibble() %>%  
arrange(r.drop)  
  
print(item_stats %>% select(Question, r.drop, raw.r))
```

```
## # A tibble: 26 × 3  
## Question r.drop raw.r  
## <chr> <dbl> <dbl>  
## 1 Q7- 0.200 0.241  
## 2 Q25 0.222 0.279  
## 3 Q13 0.231 0.288  
## 4 Q11 0.264 0.301  
## 5 Q19- 0.265 0.320  
## 6 Q12 0.273 0.308  
## 7 Q20 0.384 0.431  
## 8 Q17- 0.398 0.452  
## 9 Q15 0.461 0.509  
## 10 Q16- 0.470 0.495  
## # i 16 more rows
```

```
#Programmatic reversal of negatively correlated items  
items_to_reverse <- item_stats %>%  
filter(r.drop < 0) %>%  
pull(Question)  
  
if (length(items_to_reverse) > 0) {  
cat("\nReversing items with negative item-total correlation:",  
paste(items_to_reverse, collapse = ", "), "\n")  
  
df <- df %>%  
mutate(across(all_of(items_to_reverse), ~ -1 * .x))  
  
#Recalculate alpha after reversal  
alpha_after <- psych::alpha(df %>% select(all_of(question_cols)),  
check.keys = TRUE)  
cat("\n=== CRONBACH'S ALPHA (AFTER REVERSAL) ===\n")  
print(alpha_after$total)  
} else {  
cat("\nNo items required reversal.\n")  
}
```

```
##  
## No items required reversal.
```

```
# Party Filtering (Threshold = 13)
```

```
#Filter parties with at least 13 candidates  
threshold <- 6  
valid_parties <- party_counts %>%  
filter(n >= threshold) %>%  
pull(party)  
  
cat("Keeping parties with >=", threshold, "candidates:",  
paste(valid_parties, collapse = ", "), "\n")
```

```
## Keeping parties with >= 6 candidates: F. SF - Socialistisk Folkeparti, Ø. Enhedslisten – De Rød-Grønne, A. Socialdemokratiet, B. Radikale Venstre, C. Det Konservative Folkeparti, V. Venstre, Danmarks Liberale Parti, Æ. Danmarksdemokraterne – Inger Støjberg, I. Liberal Alliance, M. Moderaterne, O. Dansk Folkeparti, Å. Alternativet
```

```
df <- df %>%
filter(party %in% valid_parties) %>%
mutate(party = droplevels(factor(party)))

cat("Remaining observations:", nrow(df), "\n")
```

```
## Remaining observations: 141
```

```
cat("Remaining parties:", length(levels(df$party)), "\n")
```

```
## Remaining parties: 11
```

```
# Principal Component Analysis (PCA)
```

```
## PCA Computation and Variance Explained
```

```
#Prepare scaled matrix for PCA
question_matrix <- df %>%
select(all_of(question_cols)) %>%
as.matrix() %>%
scale()
```

```
#Perform PCA
pca <- prcomp(question_matrix, center = TRUE, scale. = TRUE)
```

```
#Extract eigenvalues
eig <- factoextra::get_eigenvalue(pca)
cat("\n=== VARIANCE EXPLAINED BY PRINCIPAL COMPONENTS ===\n")
```

```
##
## === VARIANCE EXPLAINED BY PRINCIPAL COMPONENTS ===
```

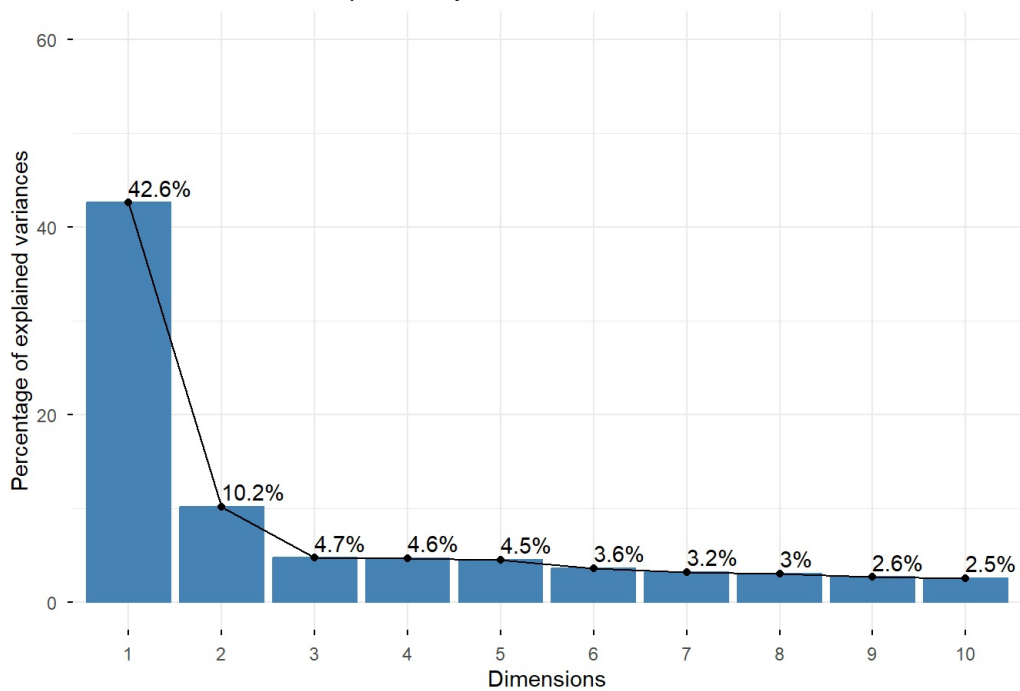
```
print(eig[1:6, ])
```

```
##      eigenvalue variance.percent cumulative.variance.percent
## Dim.1 11.0764154      42.601598      42.60160
## Dim.2  2.6455999      10.175384      52.77698
## Dim.3  1.2190605       4.688694      57.46568
## Dim.4  1.2069083       4.641955      62.10763
## Dim.5  1.1711545       4.504441      66.61207
## Dim.6  0.9361667       3.600641      70.21271
```

```
#Store PC1 and PC2 variance percentages
pc1_pct <- eig[1, "variance.percent"]
pc2_pct <- eig[2, "variance.percent"]
```

```
#Scree plot
fviz_eig(pca, addlabels = TRUE, ylim = c(0, 60)) +
labs(title = "Scree Plot: Variance Explained by Each PC")
```

Scree Plot: Variance Explained by Each PC



PCA Loadings Analysis

#Extract loadings

```
loadings <- as.data.frame(pca$rotation) %>%
  rownames_to_column("Question")
```

#Top questions for PC1 (ideological dimension)

```
top_PC1 <- loadings %>%
  arrange(desc(abs(PC1))) %>%
  slice_head(n = 15)
```

#Top questions for PC2

```
top_PC2 <- loadings %>%
  arrange(desc(abs(PC2))) %>%
  slice_head(n = 15)
```

```
cat("\n=== TOP 15 QUESTIONS DEFINING PC1 (Main Ideological Dimension) ===\n")
```

```
##
```

```
## === TOP 15 QUESTIONS DEFINING PC1 (Main Ideological Dimension) ===
```

```
print(top_PC1%>% select(Question, PC1))
```

```
##      Question      PC1
## 1      Q24 -0.2758855
## 2       Q1  0.2670380
## 3      Q26 -0.2577552
## 4       Q4 -0.2543811
## 5      Q22  0.2521825
## 6       Q8  0.2446244
## 7       Q3  0.2444561
## 8       Q6 -0.2337878
## 9       Q5  0.2328735
## 10     Q14  0.2283709
## 11      Q2  0.2282875
## 12      Q9 -0.2228678
## 13     Q10 -0.2149987
## 14     Q21  0.2006510
## 15     Q18  0.1988635
```

```
cat("\n=== TOP 15 QUESTIONS DEFINING PC2 ===\n")
```

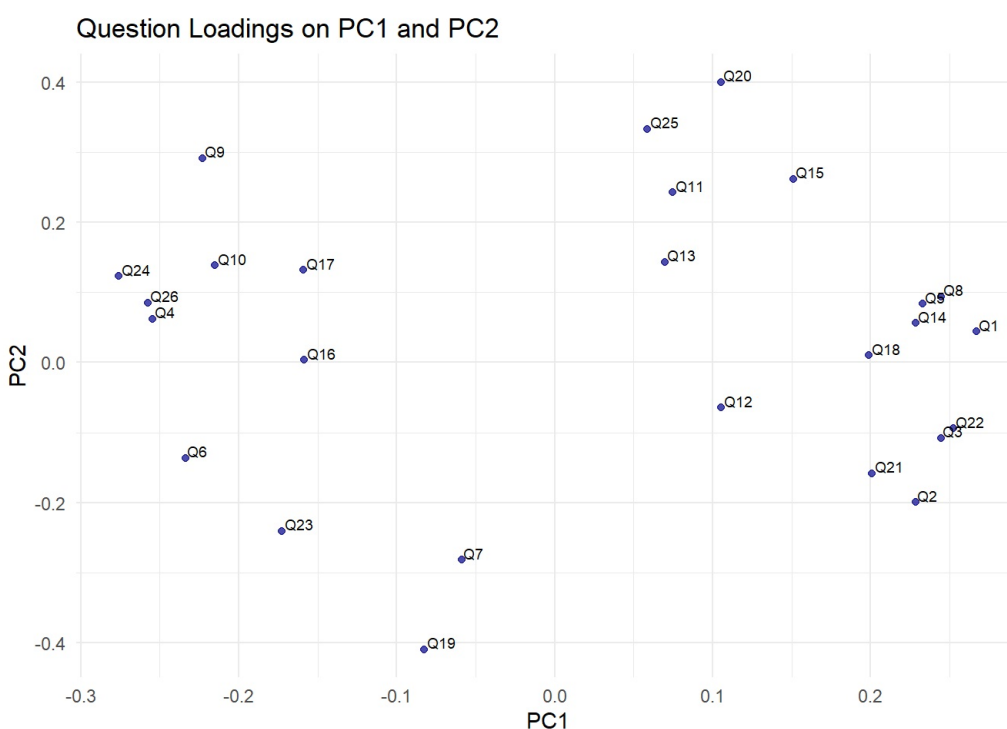
```
##
```

```
## === TOP 15 QUESTIONS DEFINING PC2 ===
```

```
print(top_PC2%>% select(Question, PC2))
```

##	Question	PC2
## 1	Q19	-0.4090145
## 2	Q20	0.3999700
## 3	Q25	0.3330235
## 4	Q9	0.2910234
## 5	Q7	-0.2816775
## 6	Q15	0.2619154
## 7	Q11	0.2426590
## 8	Q23	-0.2406361
## 9	Q2	-0.1995390
## 10	Q21	-0.1582856
## 11	Q13	0.1431456
## 12	Q10	0.1381873
## 13	Q6	-0.1363483
## 14	Q17	0.1318930
## 15	Q24	0.1230837

```
#Visualize loadings
ggplot(loadings, aes(x = PC1, y = PC2, label = Question)) +
  geom_point(alpha = 0.7, color = "darkblue") +
  geom_text(size = 2.5, hjust = -0.1, vjust = -0.1) +
  theme_minimal() +
  labs(title = "Question Loadings on PC1 and PC2")
```



Candidate Positioning in PC Space

#Extract candidate scores

```
candidate_scores <- as.data.frame(pca$x) %>%  
mutate(candidate_name = df$candidate_name,  
party = df$party)
```

#Calculate party centroids

```
party_centroids <- candidate_scores %>%  
group_by(party) %>%  
summarise(across(starts_with("PC"), mean),  
n = n(),  
.groups = "drop")
```

#Calculate distance from party centroid for each candidate

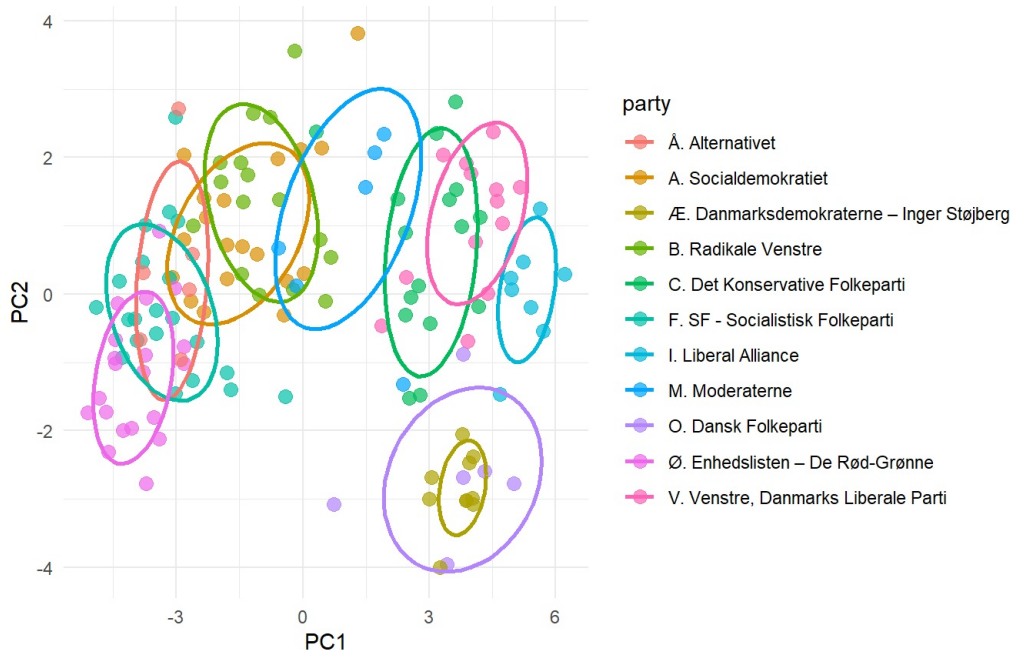
```
candidate_scores <- candidate_scores %>%  
left_join(party_centroids %>%  
select(party, PC1_cent = PC1, PC2_cent = PC2),  
by = "party") %>%  
mutate(distance_from_party = sqrt((PC1 - PC1_cent)^2 + (PC2 - PC2_cent)^2))
```

#Plot: PC1 vs PC2 with party colors and ellipses

```
groups_with_enough <- candidate_scores %>%  
count(party) %>%  
filter(n >= 3) %>%  
pull(party)  
  
ggplot(candidate_scores, aes(x = PC1, y = PC2, color = party)) +  
geom_point(size = 3, alpha = 0.7) +  
stat_ellipse(data = candidate_scores %>% filter(party %in% groups_with_enough),  
aes(group = party), level = 0.68, linewidth = 0.9) +  
theme_minimal() +  
labs(title = "Candidate Positioning in PC1-PC2 Space",  
subtitle = paste0("PC1: ", round(pc1_pct, 1), "%; PC2: ", round(pc2_pct, 1), "%"))
```

Candidate Positioning in PC1-PC2 Space

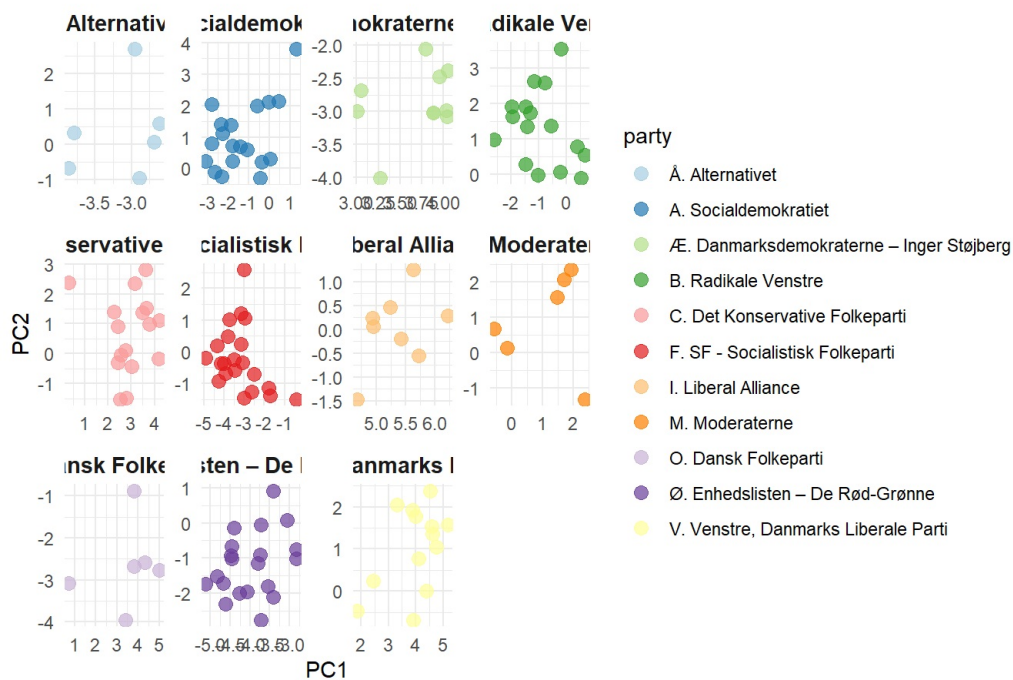
PC1: 42.6%; PC2: 10.2%



#Faceted view by party

```
ggplot(candidate_scores, aes(x = PC1, y = PC2)) +  
geom_point(aes(color = party), size = 3, alpha = 0.7) +  
facet_wrap(~party, scales = "free") +  
theme_minimal() +  
scale_color_brewer(palette = "Paired") +  
labs(title = "Ideological Spread Within Each Party") +  
theme(strip.text = element_text(face = "bold", size = 11))
```

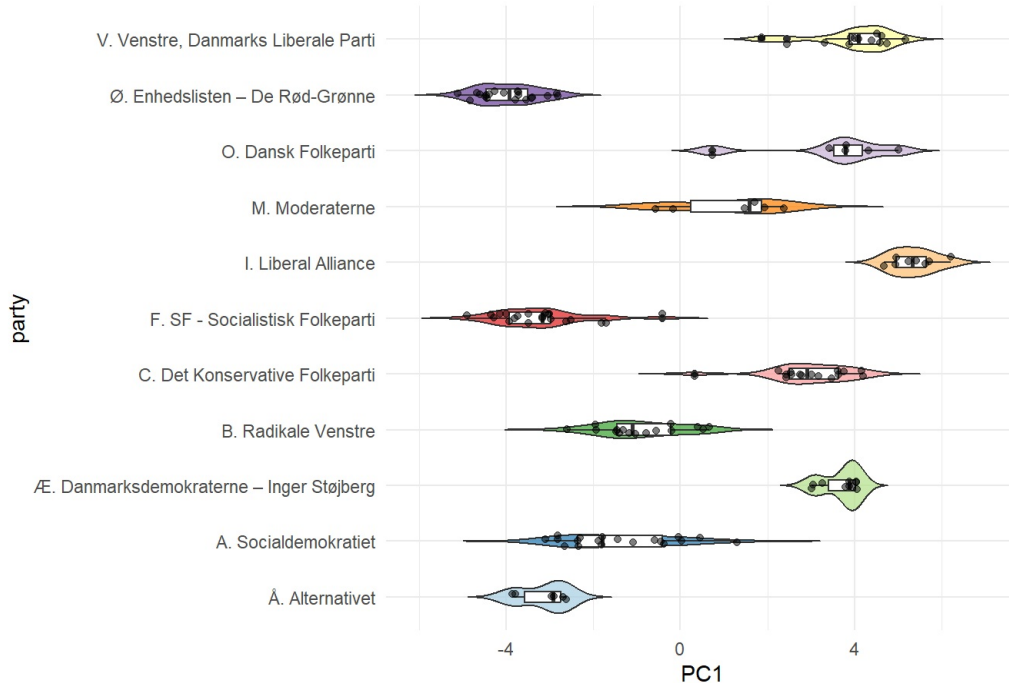
Ideological Spread Within Each Party



#Violin plot: PC1 distribution by party

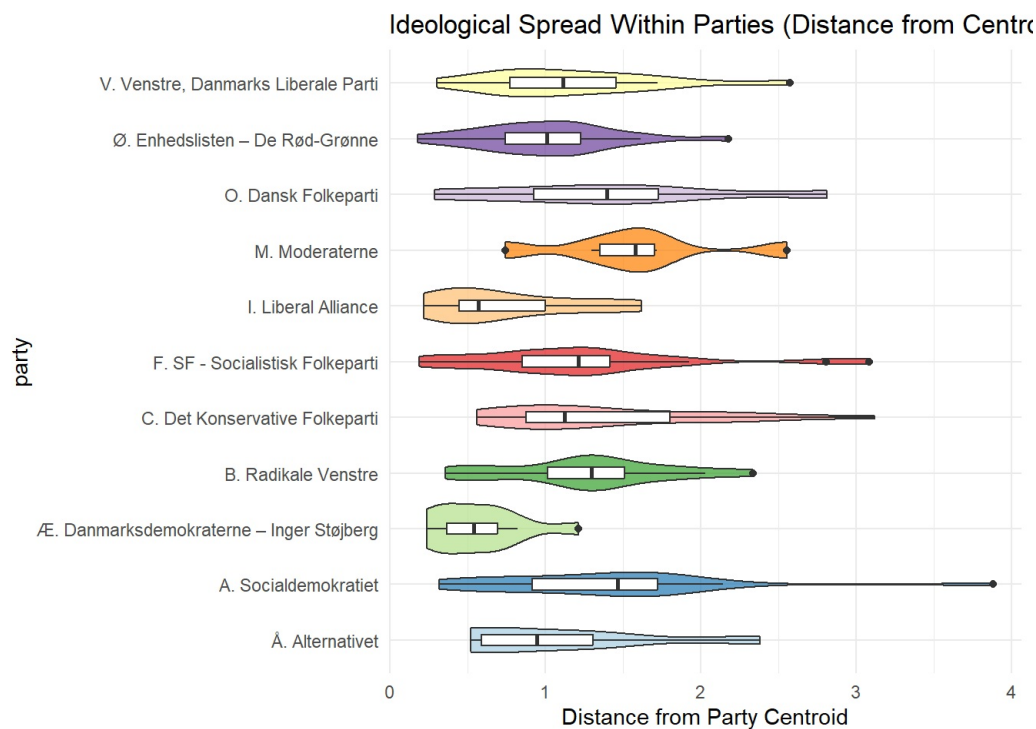
```
ggplot(candidate_scores, aes(x = party, y = PC1, fill = party)) +
  geom_violin(alpha = 0.7, trim = FALSE) +
  geom_boxplot(width = 0.2, fill = "white") +
  geom_jitter(width = 0.1, size = 1.5, alpha = 0.5) +
  coord_flip() +
  theme_minimal() +
  scale_fill_brewer(palette = "Paired") +
  labs(title = "PC1 Distribution by Party (Main Ideological Axis)") +
  theme(legend.position = "none")
```

PC1 Distribution by Party (Main Ideological Axis)



#Ideological spread within parties

```
ggplot(candidate_scores, aes(x = party, y = distance_from_party, fill = party)) +
  geom_violin(alpha = 0.7) +
  geom_boxplot(width = 0.2, fill = "white") +
  coord_flip() +
  theme_minimal() +
  scale_fill_brewer(palette = "Paired") +
  labs(title = "Ideological Spread Within Parties (Distance from Centroid)",
       y = "Distance from Party Centroid") +
  theme(legend.position = "none")
```



```
## Statistical Testing: Party Differences on PC1
```

```
#ANOVA: Do parties differ on PC1?
```

```
anova_pc1 <- aov(PC1 ~ party, data = candidate_scores)
cat("\n=== ANOVA: PC1 ~ Party ===\n")
```

```
##
## === ANOVA: PC1 ~ Party ===
```

```
print(summary(anova_pc1))
```

```
##           Df Sum Sq Mean Sq F value Pr(>F)
## party      10 1431.9  143.19   156.7 <2e-16 ***
## Residuals  130  118.8    0.91
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#Post-hoc comparisons
```

```
cat("\n=== Tukey HSD Post-Hoc Comparisons ===\n")
```

```
##
## === Tukey HSD Post-Hoc Comparisons ===
```

```
print(TukeyHSD(anova_pc1))
```

```
## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = PC1 ~ party, data = candidate_scores)
##
## $party
##
##           diff
## A. Socialdemokratiet-Å. Alternativet 1.76248932
## Æ. Danmarksdemokraterne – Inger Støjberg-Å. Alternativet 6.83251787
## B. Radikale Venstre-Å. Alternativet 2.22845534
## C. Det Konservative Folkeparti-Å. Alternativet 6.08232321
## F. SF - Socialistisk Folkeparti-Å. Alternativet -0.08692732
## I. Liberal Alliance-Å. Alternativet 8.48480789
## M. Moderaterne-Å. Alternativet 4.26023032
## O. Dansk Folkeparti-Å. Alternativet 6.65011290
## Ø. Enhedslisten – De Rød-Grønne-Å. Alternativet -0.83327823
## V. Venstre, Danmarks Liberale Parti-Å. Alternativet 7.10292373
## Æ. Danmarksdemokraterne – Inger Støjberg-A. Socialdemokratiet 5.07002855
## B. Radikale Venstre-A. Socialdemokratiet 0.46596601
## C. Det Konservative Folkeparti-A. Socialdemokratiet 4.31983388
## F. SF - Socialistisk Folkeparti-A. Socialdemokratiet -1.84941665
```

## I. Liberal Alliance-A. Socialdemokratiet	6.72231857
## M. Moderaterne-A. Socialdemokratiet	2.49774099
## O. Dansk Folkeparti-A. Socialdemokratiet	4.88762357
## Ø. Enhedslisten – De Rød-Grønne-A. Socialdemokratiet	-2.59576755
## V. Venstre, Danmarks Liberale Parti-A. Socialdemokratiet	5.34043441
## B. Radikale Venstre-Æ. Danmarksdemokraterne – Inger Støjberg	-4.60406254
## C. Det Konservative Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-0.75019467
## F. SF - Socialistisk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-6.91944520
## I. Liberal Alliance-Æ. Danmarksdemokraterne – Inger Støjberg	1.65229002
## M. Moderaterne-Æ. Danmarksdemokraterne – Inger Støjberg	-2.57228756
## O. Dansk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-0.18240498
## Ø. Enhedslisten – De Rød-Grønne-Æ. Danmarksdemokraterne – Inger Støjberg	-7.66579610
## V. Venstre, Danmarks Liberale Parti-Æ. Danmarksdemokraterne – Inger Støjberg	0.27040586
## C. Det Konservative Folkeparti-B. Radikale Venstre	3.85386787
## F. SF - Socialistisk Folkeparti-B. Radikale Venstre	-2.31538266
## I. Liberal Alliance-B. Radikale Venstre	6.25635255
## M. Moderaterne-B. Radikale Venstre	2.03177498
## O. Dansk Folkeparti-B. Radikale Venstre	4.42165756
## Ø. Enhedslisten – De Rød-Grønne-B. Radikale Venstre	-3.06173356
## V. Venstre, Danmarks Liberale Parti-B. Radikale Venstre	4.87446839
## F. SF - Socialistisk Folkeparti-C. Det Konservative Folkeparti	-6.16925053
## I. Liberal Alliance-C. Det Konservative Folkeparti	2.40248468
## M. Moderaterne-C. Det Konservative Folkeparti	-1.82209289
## O. Dansk Folkeparti-C. Det Konservative Folkeparti	0.56778969
## Ø. Enhedslisten – De Rød-Grønne-C. Det Konservative Folkeparti	-6.91560143
## V. Venstre, Danmarks Liberale Parti-C. Det Konservative Folkeparti	1.02060052
## I. Liberal Alliance-F. SF - Socialistisk Folkeparti	8.57173521
## M. Moderaterne-F. SF - Socialistisk Folkeparti	4.34715764
## O. Dansk Folkeparti-F. SF - Socialistisk Folkeparti	6.73704022
## Ø. Enhedslisten – De Rød-Grønne-F. SF - Socialistisk Folkeparti	-0.74635090
## V. Venstre, Danmarks Liberale Parti-F. SF - Socialistisk Folkeparti	7.18985105
## M. Moderaterne-I. Liberal Alliance	-4.22457757
## O. Dansk Folkeparti-I. Liberal Alliance	-1.83469500
## Ø. Enhedslisten – De Rød-Grønne-I. Liberal Alliance	-9.31808612
## V. Venstre, Danmarks Liberale Parti-I. Liberal Alliance	-1.38188416
## O. Dansk Folkeparti-M. Moderaterne	2.38988258
## Ø. Enhedslisten – De Rød-Grønne-M. Moderaterne	-5.09350855
## V. Venstre, Danmarks Liberale Parti-M. Moderaterne	2.84269341
## Ø. Enhedslisten – De Rød-Grønne-O. Dansk Folkeparti	-7.48339112
## V. Venstre, Danmarks Liberale Parti-O. Dansk Folkeparti	0.45281084
## V. Venstre, Danmarks Liberale Parti-Ø. Enhedslisten – De Rød-Grønne	7.93620196
##	lwr
## A. Socialdemokratiet-Å. Alternativet	0.2957491
## Æ. Danmarksdemokraterne – Inger Støjberg-Å. Alternativet	5.2151084
## B. Radikale Venstre-Å. Alternativet	0.7290766
## C. Det Konservative Folkeparti-Å. Alternativet	4.5829445
## F. SF - Socialistisk Folkeparti-Å. Alternativet	-1.5368079
## I. Liberal Alliance-Å. Alternativet	6.7932805
## M. Moderaterne-Å. Alternativet	2.4519115
## O. Dansk Folkeparti-Å. Alternativet	4.8417941
## Ø. Enhedslisten – De Rød-Grønne-Å. Alternativet	-2.2911914
## V. Venstre, Danmarks Liberale Parti-Å. Alternativet	5.5570817
## Æ. Danmarksdemokraterne – Inger Støjberg-A. Socialdemokratiet	3.8463768
## B. Radikale Venstre-A. Socialdemokratiet	-0.5967882
## C. Det Konservative Folkeparti-A. Socialdemokratiet	3.2570796
## F. SF - Socialistisk Folkeparti-A. Socialdemokratiet	-2.8411140
## I. Liberal Alliance-A. Socialdemokratiet	5.4022524
## M. Moderaterne-A. Socialdemokratiet	1.0310008
## O. Dansk Folkeparti-A. Socialdemokratiet	3.4208833
## Ø. Enhedslisten – De Rød-Grønne-A. Socialdemokratiet	-3.5991722
## V. Venstre, Danmarks Liberale Parti-A. Socialdemokratiet	4.2130757
## B. Radikale Venstre-Æ. Danmarksdemokraterne – Inger Støjberg	-5.8666524
## C. Det Konservative Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-2.0127846
## F. SF - Socialistisk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-8.1228364
## I. Liberal Alliance-Æ. Danmarksdemokraterne – Inger Støjberg	0.1666045
## M. Moderaterne-Æ. Danmarksdemokraterne – Inger Støjberg	-4.1896971
## O. Dansk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-1.7998145
## Ø. Enhedslisten – De Rød-Grønne-Æ. Danmarksdemokraterne – Inger Støjberg	-8.8788532
## V. Venstre, Danmarks Liberale Parti-Æ. Danmarksdemokraterne – Inger Støjberg	-1.0470251
## C. Det Konservative Folkeparti-B. Radikale Venstre	2.7465033
## F. SF - Socialistisk Folkeparti-B. Radikale Venstre	-3.3547447
## I. Liberal Alliance-B. Radikale Venstre	4.9001135
## M. Moderaterne-B. Radikale Venstre	0.5323962
## O. Dansk Folkeparti-B. Radikale Venstre	2.9222788
## Ø. Enhedslisten – De Rød-Grønne-B. Radikale Venstre	-4.1122718
## V. Venstre, Danmarks Liberale Parti-B. Radikale Venstre	3.7049610
## F. SF - Socialistisk Folkeparti-C. Det Konservative Folkeparti	-7.2086126
## I. Liberal Alliance-C. Det Konservative Folkeparti	1.0462456
## M. Moderaterne-C. Det Konservative Folkeparti	-3.3214716

## O. Dansk Folkeparti-C. Det Konservative Folkeparti	-0.9315890
## Ø. Enhedslisten – De Rød-Grønne-C. Det Konservative Folkeparti	-7.9661397
## V. Venstre, Danmarks Liberale Parti-C. Det Konservative Folkeparti	-0.1489068
## I. Liberal Alliance-F. SF - Socialistisk Folkeparti	7.2704276
## M. Moderaterne-F. SF - Socialistisk Folkeparti	2.8972771
## O. Dansk Folkeparti-F. SF - Socialistisk Folkeparti	5.2871596
## Ø. Enhedslisten – De Rød-Grønne-F. SF - Socialistisk Folkeparti	-1.7249457
## V. Venstre, Danmarks Liberale Parti-F. SF - Socialistisk Folkeparti	6.0845165
## M. Moderaterne-I. Liberal Alliance	-5.9161049
## O. Dansk Folkeparti-I. Liberal Alliance	-3.5262223
## Ø. Enhedslisten – De Rød-Grønne-I. Liberal Alliance	-10.6283376
## V. Venstre, Danmarks Liberale Parti-I. Liberal Alliance	-2.7893200
## O. Dansk Folkeparti-M. Moderaterne	0.5815638
## Ø. Enhedslisten – De Rød-Grønne-M. Moderaterne	-6.5514218
## V. Venstre, Danmarks Liberale Parti-M. Moderaterne	1.2968513
## Ø. Enhedslisten – De Rød-Grønne-O. Dansk Folkeparti	-8.9413043
## V. Venstre, Danmarks Liberale Parti-O. Dansk Folkeparti	-1.0930312
## V. Venstre, Danmarks Liberale Parti-Ø. Enhedslisten – De Rød-Grønne	6.8203517
##	upr
## A. Socialdemokratiet-Å. Alternativet	3.22922955
## Æ. Danmarksdemokraterne – Inger Støjberg-Å. Alternativet	8.44992737
## B. Radikale Venstre-Å. Alternativet	3.72783407
## C. Det Konservative Folkeparti-Å. Alternativet	7.58170194
## F. SF - Socialistisk Folkeparti-Å. Alternativet	1.36295326
## I. Liberal Alliance-Å. Alternativet	10.17633523
## M. Moderaterne-Å. Alternativet	6.06854911
## O. Dansk Folkeparti-Å. Alternativet	8.45843169
## Ø. Enhedslisten – De Rød-Grønne-Å. Alternativet	0.62463499
## V. Venstre, Danmarks Liberale Parti-Å. Alternativet	8.64876580
## Æ. Danmarksdemokraterne – Inger Støjberg-A. Socialdemokratiet	6.29368026
## B. Radikale Venstre-A. Socialdemokratiet	1.52872027
## C. Det Konservative Folkeparti-A. Socialdemokratiet	5.38258814
## F. SF - Socialistisk Folkeparti-A. Socialdemokratiet	-0.85771926
## I. Liberal Alliance-A. Socialdemokratiet	8.04238477
## M. Moderaterne-A. Socialdemokratiet	3.96448122
## O. Dansk Folkeparti-A. Socialdemokratiet	6.35436379
## Ø. Enhedslisten – De Rød-Grønne-A. Socialdemokratiet	-1.59236286
## V. Venstre, Danmarks Liberale Parti-A. Socialdemokratiet	6.46779314
## B. Radikale Venstre-Æ. Danmarksdemokraterne – Inger Støjberg	-3.34147265
## C. Det Konservative Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	0.51239522
## F. SF - Socialistisk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	-5.71605402
## I. Liberal Alliance-Æ. Danmarksdemokraterne – Inger Støjberg	3.13797551
## M. Moderaterne-Æ. Danmarksdemokraterne – Inger Støjberg	-0.95487806
## O. Dansk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	1.43500452
## Ø. Enhedslisten – De Rød-Grønne-Æ. Danmarksdemokraterne – Inger Støjberg	-6.45273898
## V. Venstre, Danmarks Liberale Parti-Æ. Danmarksdemokraterne – Inger Støjberg	1.58783682
## C. Det Konservative Folkeparti-B. Radikale Venstre	4.96123245
## F. SF - Socialistisk Folkeparti-B. Radikale Venstre	-1.27602063
## I. Liberal Alliance-B. Radikale Venstre	7.61259165
## M. Moderaterne-B. Radikale Venstre	3.53115372
## O. Dansk Folkeparti-B. Radikale Venstre	5.92103629
## Ø. Enhedslisten – De Rød-Grønne-B. Radikale Venstre	-2.01119528
## V. Venstre, Danmarks Liberale Parti-B. Radikale Venstre	6.04397574
## F. SF - Socialistisk Folkeparti-C. Det Konservative Folkeparti	-5.12988850
## I. Liberal Alliance-C. Det Konservative Folkeparti	3.75872378
## M. Moderaterne-C. Det Konservative Folkeparti	-0.32271415
## O. Dansk Folkeparti-C. Det Konservative Folkeparti	2.06716842
## Ø. Enhedslisten – De Rød-Grønne-C. Det Konservative Folkeparti	-5.86506315
## V. Venstre, Danmarks Liberale Parti-C. Det Konservative Folkeparti	2.19010787
## I. Liberal Alliance-F. SF - Socialistisk Folkeparti	9.87304286
## M. Moderaterne-F. SF - Socialistisk Folkeparti	5.79703822
## O. Dansk Folkeparti-F. SF - Socialistisk Folkeparti	8.18692080
## Ø. Enhedslisten – De Rød-Grønne-F. SF - Socialistisk Folkeparti	0.23224390
## V. Venstre, Danmarks Liberale Parti-F. SF - Socialistisk Folkeparti	8.29518564
## M. Moderaterne-I. Liberal Alliance	-2.53305023
## O. Dansk Folkeparti-I. Liberal Alliance	-0.14316765
## Ø. Enhedslisten – De Rød-Grønne-I. Liberal Alliance	-8.00783467
## V. Venstre, Danmarks Liberale Parti-I. Liberal Alliance	0.02555166
## O. Dansk Folkeparti-M. Moderaterne	4.19820137
## Ø. Enhedslisten – De Rød-Grønne-M. Moderaterne	-3.63559533
## V. Venstre, Danmarks Liberale Parti-M. Moderaterne	4.38853548
## Ø. Enhedslisten – De Rød-Grønne-O. Dansk Folkeparti	-6.02547790
## V. Venstre, Danmarks Liberale Parti-O. Dansk Folkeparti	1.99865290
## V. Venstre, Danmarks Liberale Parti-Ø. Enhedslisten – De Rød-Grønne	9.05205222
##	p adj
## A. Socialdemokratiet-Å. Alternativet	0.0060670
## Æ. Danmarksdemokraterne – Inger Støjberg-Å. Alternativet	0.0000000
## B. Radikale Venstre-Å. Alternativet	0.0001639
## C. Det Konservative Folkeparti-Å. Alternativet	0.0000000

## F. SF - Socialistisk Folkeparti-Å. Alternativet	1.0000000
## I. Liberal Alliance-Å. Alternativet	0.0000000
## M. Moderaterne-Å. Alternativet	0.0000000
## O. Dansk Folkeparti-Å. Alternativet	0.0000000
## Ø. Enhedslisten – De Rød-Grønne-Å. Alternativet	0.7335276
## V. Venstre, Danmarks Liberale Parti-Å. Alternativet	0.0000000
## Æ. Danmarksdemokraterne – Inger Støjberg-A. Socialdemokratiet	0.0000000
## B. Radikale Venstre-A. Socialdemokratiet	0.9367083
## C. Det Konservative Folkeparti-A. Socialdemokratiet	0.0000000
## F. SF - Socialistisk Folkeparti-A. Socialdemokratiet	0.0000006
## I. Liberal Alliance-A. Socialdemokratiet	0.0000000
## M. Moderaterne-A. Socialdemokratiet	0.0000072
## O. Dansk Folkeparti-A. Socialdemokratiet	0.0000000
## Ø. Enhedslisten – De Rød-Grønne-A. Socialdemokratiet	0.0000000
## V. Venstre, Danmarks Liberale Parti-A. Socialdemokratiet	0.0000000
## B. Radikale Venstre-Æ. Danmarksdemokraterne – Inger Støjberg	0.0000000
## C. Det Konservative Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	0.6852297
## F. SF - Socialistisk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	0.0000000
## I. Liberal Alliance-Æ. Danmarksdemokraterne – Inger Støjberg	0.0163164
## M. Moderaterne-Æ. Danmarksdemokraterne – Inger Støjberg	0.0000377
## O. Dansk Folkeparti-Æ. Danmarksdemokraterne – Inger Støjberg	0.9999995
## Ø. Enhedslisten – De Rød-Grønne-Æ. Danmarksdemokraterne – Inger Støjberg	0.0000000
## V. Venstre, Danmarks Liberale Parti-Æ. Danmarksdemokraterne – Inger Støjberg	0.9998510
## C. Det Konservative Folkeparti-B. Radikale Venstre	0.0000000
## F. SF - Socialistisk Folkeparti-B. Radikale Venstre	0.0000000
## I. Liberal Alliance-B. Radikale Venstre	0.0000000
## M. Moderaterne-B. Radikale Venstre	0.0009377
## O. Dansk Folkeparti-B. Radikale Venstre	0.0000000
## Ø. Enhedslisten – De Rød-Grønne-B. Radikale Venstre	0.0000000
## V. Venstre, Danmarks Liberale Parti-B. Radikale Venstre	0.0000000
## F. SF - Socialistisk Folkeparti-C. Det Konservative Folkeparti	0.0000000
## I. Liberal Alliance-C. Det Konservative Folkeparti	0.0000025
## M. Moderaterne-C. Det Konservative Folkeparti	0.0051851
## O. Dansk Folkeparti-C. Det Konservative Folkeparti	0.9764208
## Ø. Enhedslisten – De Rød-Grønne-C. Det Konservative Folkeparti	0.0000000
## V. Venstre, Danmarks Liberale Parti-C. Det Konservative Folkeparti	0.1480961
## I. Liberal Alliance-F. SF - Socialistisk Folkeparti	0.0000000
## M. Moderaterne-F. SF - Socialistisk Folkeparti	0.0000000
## O. Dansk Folkeparti-F. SF - Socialistisk Folkeparti	0.0000000
## Ø. Enhedslisten – De Rød-Grønne-F. SF - Socialistisk Folkeparti	0.3148615
## V. Venstre, Danmarks Liberale Parti-F. SF - Socialistisk Folkeparti	0.0000000
## M. Moderaterne-I. Liberal Alliance	0.0000000
## O. Dansk Folkeparti-I. Liberal Alliance	0.0217528
## Ø. Enhedslisten – De Rød-Grønne-I. Liberal Alliance	0.0000000
## V. Venstre, Danmarks Liberale Parti-I. Liberal Alliance	0.0591211
## O. Dansk Folkeparti-M. Moderaterne	0.0014333
## Ø. Enhedslisten – De Rød-Grønne-M. Moderaterne	0.0000000
## V. Venstre, Danmarks Liberale Parti-M. Moderaterne	0.0000009
## Ø. Enhedslisten – De Rød-Grønne-O. Dansk Folkeparti	0.0000000
## V. Venstre, Danmarks Liberale Parti-O. Dansk Folkeparti	0.9967332
## V. Venstre, Danmarks Liberale Parti-Ø. Enhedslisten – De Rød-Grønne	0.0000000

```
#Check homogeneity of variance
if (requireNamespace("car", quietly = TRUE)){
cat("\n=== Levene's Test for Homogeneity of Variance ===\n")
print(car::leveneTest(PC1 ~ party, data = candidate_scores))
}
```

```
##
## === Levene's Test for Homogeneity of Variance ===
## Levene's Test for Homogeneity of Variance (center = median)
##      Df F value Pr(>F)
## group 10  1.5227 0.1382
##      130
```

```
# Clustering Analysis
```

```
## Optimal Number of Clusters (Candidate-Level)
```

```
#Use first 6 PCs for clustering
```

```
pc_matrix <- candidate_scores %>%
```

```
select(starts_with("PC")) %>%
```

```
select(PC1:PC6) %>%
```

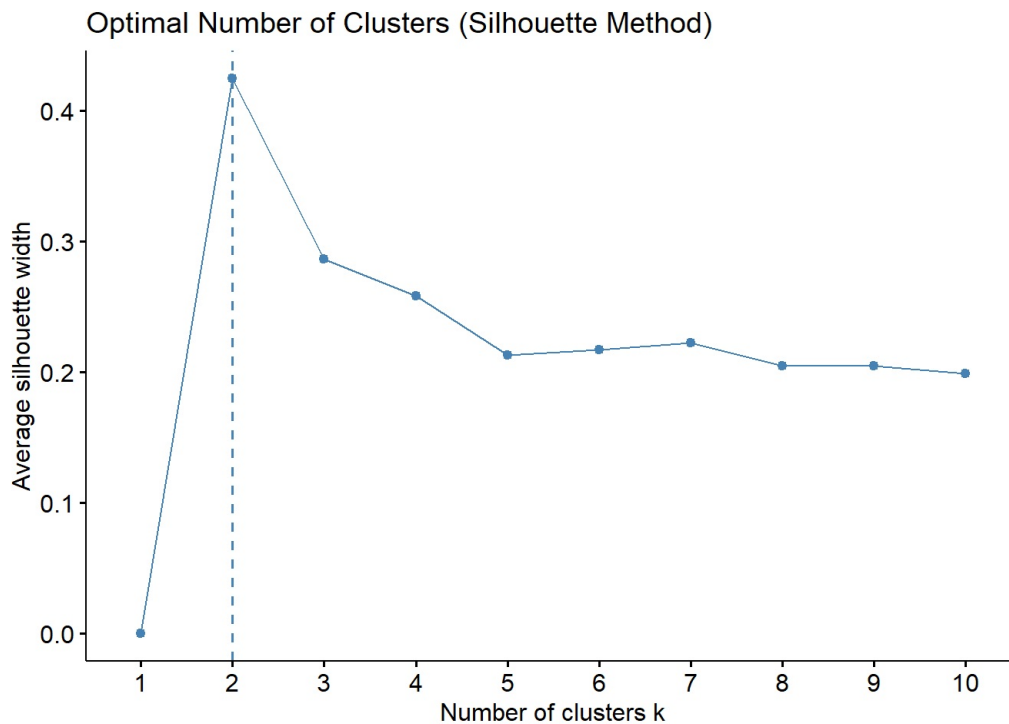
```
as.matrix()
```

```
#Determine optimal k using silhouette method
```

```
k_max <- min(10, nrow(pc_matrix) - 1)
```

```
fviz_nbclust(pc_matrix, FUN = kmeans, method = "silhouette", k.max = k_max) +
```

```
labs(title = "Optimal Number of Clusters (Silhouette Method)")
```



```
#Perform k-means with k=2 (based on typical left-right split)
```

```
k_cand <- 2
```

```
set.seed(123)
```

```
kmeans_model <- kmeans(pc_matrix, centers = k_cand, nstart = 50)
```

```
candidate_scores$cluster_cand <- factor(kmeans_model$cluster)
```

```
#Silhouette diagnostic
```

```
sil <- silhouette(as.numeric(candidate_scores$cluster_cand), dist(pc_matrix))
```

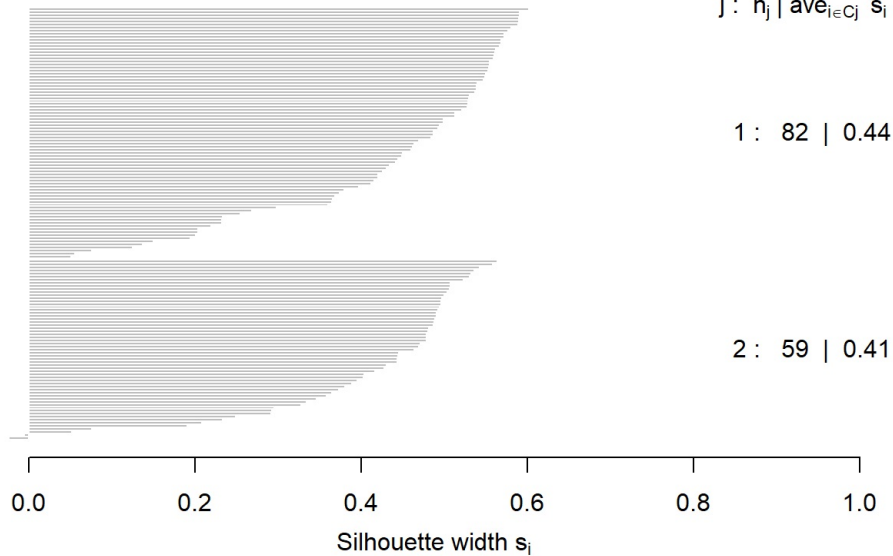
```
plot(sil, main = "Silhouette Plot for Candidate Clusters")
```

Silhouette Plot for Candidate Clusters

n = 141

2 clusters C_j

$j : n_j \mid \text{ave}_{i \in C_j} s_i$



```
#Cluster composition by party
cat("\n=== Cluster Composition by Party ===\n")
```

```
##
## === Cluster Composition by Party ===
```

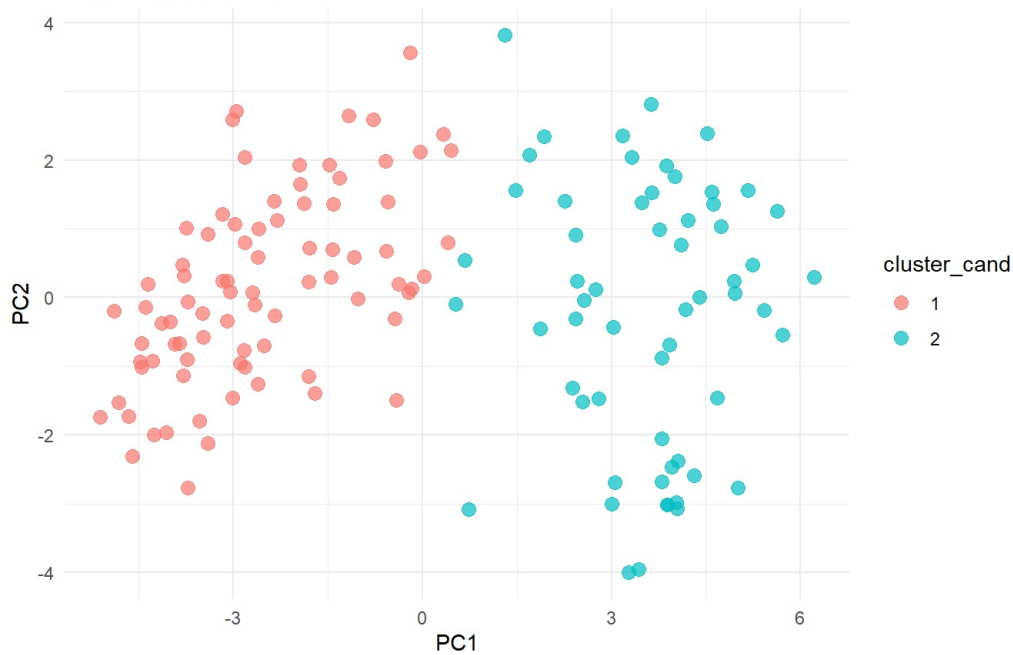
```
print(table(candidate_scores$cluster_cand, candidate_scores$party))
```

```
##
##      Å. Alternativet A. Socialdemokratiet
## 1          6          18
## 2          0           1
##
##      Æ. Danmarksdemokraterne – Inger Støjberg B. Radikale Venstre
## 1          0          14
## 2         10           2
##
##      C. Det Konservative Folkeparti F. SF - Socialistisk Folkeparti
## 1          1          21
## 2         15           0
##
##      I. Liberal Alliance M. Moderaterne O. Dansk Folkeparti
## 1          0           2           0
## 2          8           4           6
##
##      Ø. Enhedslisten – De Rød-Grønne V. Venstre, Danmarks Liberale Parti
## 1         20           0
## 2          0          13
```

```
#Visualize clusters
ggplot(candidate_scores, aes(x = PC1, y = PC2, color = cluster_cand)) +
  geom_point(size = 3, alpha = 0.7) +
  theme_minimal() +
  labs(title = paste("K-means Clustering (k =", k_cand, ")"),
       subtitle = "Clusters based on PC1-PC6")
```

K-means Clustering (k = 2)

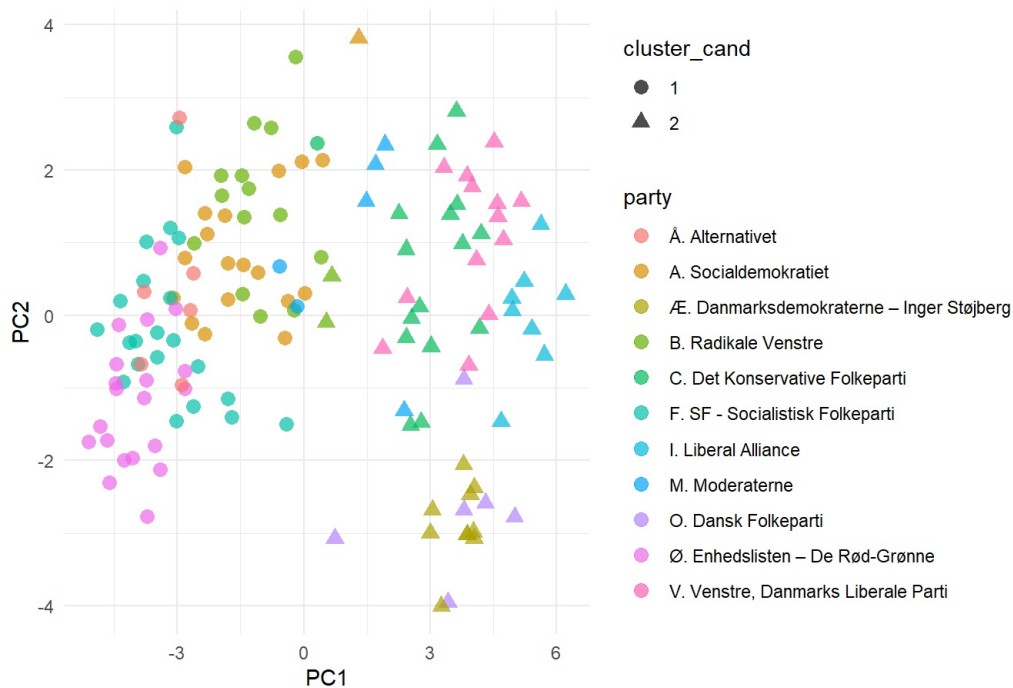
Clusters based on PC1-PC6



#Clusters with party overlaid

```
ggplot(candidate_scores, aes(x = PC1, y = PC2, color = party, shape = cluster_cand)) +
  geom_point(size = 3, alpha = 0.7) +
  theme_minimal() +
  labs(title = paste("Clusters with Party Labels (k =", k_cand, ")"))
```

Clusters with Party Labels (k = 2)



Jaccard Stability Analysis

#Bootstrap stability of clusters using Jaccard index

```
set.seed(123)
clust_boot <- fpc::clusterboot(pc_matrix, B = 100,
  clustermethod = kmeansCBI,
  krange = k_cand,
  seed = 123)
```

```
## boot 1
## boot 2
## boot 3
## boot 4
## boot 5
## boot 6
## boot 7
```

```
## boot 8
## boot 9
## boot 10
## boot 11
## boot 12
## boot 13
## boot 14
## boot 15
## boot 16
## boot 17
## boot 18
## boot 19
## boot 20
## boot 21
## boot 22
## boot 23
## boot 24
## boot 25
## boot 26
## boot 27
## boot 28
## boot 29
## boot 30
## boot 31
## boot 32
## boot 33
## boot 34
## boot 35
## boot 36
## boot 37
## boot 38
## boot 39
## boot 40
## boot 41
## boot 42
## boot 43
## boot 44
## boot 45
## boot 46
## boot 47
## boot 48
## boot 49
## boot 50
## boot 51
## boot 52
## boot 53
## boot 54
## boot 55
## boot 56
## boot 57
## boot 58
## boot 59
## boot 60
## boot 61
## boot 62
## boot 63
## boot 64
## boot 65
## boot 66
## boot 67
## boot 68
## boot 69
## boot 70
## boot 71
## boot 72
## boot 73
## boot 74
## boot 75
## boot 76
## boot 77
## boot 78
## boot 79
## boot 80
## boot 81
## boot 82
## boot 83
## boot 84
## boot 85
## boot 86
```

```
## boot 87
## boot 88
## boot 89
## boot 90
## boot 91
## boot 92
## boot 93
## boot 94
## boot 95
## boot 96
## boot 97
## boot 98
## boot 99
## boot 100
```

```
cat("\n=== Jaccard Stability (Bootstrap) ===\n")
```

```
##
## === Jaccard Stability (Bootstrap) ===
```

```
cat("Mean Jaccard coefficient per cluster:\n")
```

```
## Mean Jaccard coefficient per cluster:
```

```
print(clust_boot$bootmean)
```

```
## [1] 0.9836678 0.9775980
```

```
cat("\nNote: Values > 0.75 indicate stable clusters\n")
```

```
##
## Note: Values > 0.75 indicate stable clusters
```

```
# Predictive Modeling with Nested CV (NO DATA LEAKAGE)

## Prepare Full Dataset for CV

#Use entire filtered dataset (n=105) with cross-validation
#NO train/test split - CV provides unbiased estimates
X_full <- df %>% select(all_of(question_cols))
y_full_pretty <- factor(df$party)
y_full <- make_safe_levels(y_full_pretty)

cat("Total observations for nested CV:", nrow(X_full), "\n")
```

```
## Total observations for nested CV: 141
```

```
cat("Number of classes:", length(levels(y_full)), "\n")
```

```
## Number of classes: 11
```

```
cat("Observations per class:\n")
```

```
## Observations per class:
```

```
print(table(y_full_pretty))
```

```
## y_full_pretty
##                Å. Alternativet
##                6
##                A. Socialdemokratiet
##                19
## Æ. Danmarksdemokraterne – Inger Støjberg
##                10
##                B. Radikale Venstre
##                16
##                C. Det Konservative Folkeparti
##                16
##                F. SF - Socialistisk Folkeparti
##                21
##                I. Liberal Alliance
##                8
##                M. Moderaterne
##                6
##                O. Dansk Folkeparti
##                6
##                Ø. Enhedslisten – De Rød-Grønne
##                20
##                V. Venstre, Danmarks Liberale Parti
##                13
```

```
## Outer CV Loop: Performance Estimation
```

```
#Outer loop: 10-fold CV repeated 5 times for stable performance estimates
```

```
#This gives us 50 performance measurements per model
```

```
outer_ctrl <- trainControl(
  method = "repeatedcv",
  number = 10, # 10-fold CV (each fold ~10-11 observations)
  repeats = 5, # Repeat 5 times for stability
  classProbs = TRUE,
  summaryFunction = defaultSummary, # Better metrics for multi-class
  savePredictions = "final",
  verboseIter = FALSE
)
```

```
cat("Outer CV configuration:\n")
```

```
## Outer CV configuration:
```

```
cat(" Method: 10-fold CV repeated 5 times\n")
```

```
## Method: 10-fold CV repeated 5 times
```

```
cat(" Total model fits per algorithm: 50\n")
```

```
## Total model fits per algorithm: 50
```

```
cat(" Each test fold: ~", round(nrow(X_full)/10), "observations\n")
```

```
## Each test fold: ~ 14 observations
```

```
## Train Models with Nested CV
```

```
#Random Forest with nested CV
```

```
set.seed(123)
rf_cv <- train(
  x = X_full,
  y = y_full,
  method = "rf",
  trControl = outer_ctrl,
  tuneLength = 5,
  metric = "Accuracy",
  importance = TRUE
)
```

```
cat("\n=== RANDOM FOREST: 10-FOLD CV (REPEATED 5x) ===\n")
```

```
##  
## === RANDOM FOREST: 10-FOLD CV (REPEATED 5x) ===
```

```
print(rf_cv)
```

```
## Random Forest  
##  
## 141 samples  
## 26 predictor  
## 11 classes: 'Å..Alternativet', 'A..Socialdemokratiet', 'Æ..Danmarksdemokraterne...Inger.Støjberg', 'B..Radika  
le.Venstre', 'C..Det.Konservative.Folkeparti', 'F..SF...Socialistisk.Folkeparti', 'I..Liberal.Alliance', 'M..Mode  
raterne', 'O..Dansk.Folkeparti', 'Ø..Enhedslisten...De.Rød.Grønne', 'V..Venstre..Danmarks.Liberale.Parti'  
##  
## No pre-processing  
## Resampling: Cross-Validated (10 fold, repeated 5 times)  
## Summary of sample sizes: 127, 127, 125, 128, 126, 127, ...  
## Resampling results across tuning parameters:  
##  
##   mtry  Accuracy   Kappa  
##    2    0.7100639 0.6710388  
##    8    0.6622884 0.6170494  
##   14    0.6333584 0.5842557  
##   20    0.6359272 0.5875748  
##   26    0.6158145 0.5650804  
##  
## Accuracy was used to select the optimal model using the largest value.  
## The final value used for the model was mtry = 2.
```

```
cat("\nBest mtry:", rf_cv$bestTune$mtry, "\n")
```

```
##  
## Best mtry: 2
```

```
#SVM Radial  
set.seed(123)  
svm_cv <- train(  
  x = X_full,  
  y = y_full,  
  method = "svmRadial",  
  preProcess = c("center", "scale"),  
  trControl = outer_ctrl,  
  tuneLength = 5,  
  metric = "Accuracy"  
)  
  
cat("\n=== SVM RADIAL: 10-FOLD CV (REPEATED 5x) ===\n")
```

```
##  
## === SVM RADIAL: 10-FOLD CV (REPEATED 5x) ===
```

```
print(svm_cv)
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 141 samples
## 26 predictor
## 11 classes: 'Å..Alternativet', 'A..Socialdemokratiet', 'Æ..Danmarksdemokraterne...Inger.Støjberg', 'B..Radika
le.Venstre', 'C..Det.Konservative.Folkeparti', 'F..SF...Socialistisk.Folkeparti', 'I..Liberal.Alliance', 'M..Mode
raterne', 'O..Dansk.Folkeparti', 'Ø..Enhedslisten...De.Rød.Grønne', 'V..Venstre..Danmarks.Liberale.Parti'
##
## Pre-processing: centered (26), scaled (26)
## Resampling: Cross-Validated (10 fold, repeated 5 times)
## Summary of sample sizes: 127, 127, 125, 128, 126, 127, ...
## Resampling results across tuning parameters:
##
##      C      Accuracy      Kappa
## 0.25 0.3628861 0.2982454
## 0.50 0.3801383 0.3181434
## 1.00 0.3977224 0.3333090
## 2.00 0.3891958 0.3223613
## 4.00 0.3854312 0.3179815
##
## Tuning parameter 'sigma' was held constant at a value of 0.0267074
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.0267074 and C = 1.
```

```
#Multinomial Logistic Regression
set.seed(123)
lr_cv <- train(
  x = X_full,
  y = y_full,
  method = "multinom",
  preProcess = c("center", "scale"),
  trControl = outer_ctrl,
  tuneLength = 3,
  metric = "Accuracy",
  trace = FALSE
)

cat("\n=== MULTINOMIAL LOGISTIC REGRESSION: 10-FOLD CV (REPEATED 5x) ===\n")
```

```
##
## === MULTINOMIAL LOGISTIC REGRESSION: 10-FOLD CV (REPEATED 5x) ===
```

```
print(lr_cv)
```

```
## Penalized Multinomial Regression
##
## 141 samples
## 26 predictor
## 11 classes: 'Å..Alternativet', 'A..Socialdemokratiet', 'Æ..Danmarksdemokraterne...Inger.Støjberg', 'B..Radika
le.Venstre', 'C..Det.Konservative.Folkeparti', 'F..SF...Socialistisk.Folkeparti', 'I..Liberal.Alliance', 'M..Mode
raterne', 'O..Dansk.Folkeparti', 'Ø..Enhedslisten...De.Rød.Grønne', 'V..Venstre..Danmarks.Liberale.Parti'
##
## Pre-processing: centered (26), scaled (26)
## Resampling: Cross-Validated (10 fold, repeated 5 times)
## Summary of sample sizes: 127, 127, 125, 128, 126, 127, ...
## Resampling results across tuning parameters:
##
##      decay Accuracy      Kappa
## 0e+00 0.4424747 0.3722067
## 1e-04 0.5657889 0.5100556
## 1e-01 0.6168022 0.5675973
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was decay = 0.1.
```

```
## Model Comparison

#Compare all models
results <- resamples(list(
  RandomForest = rf_cv,
  SVM = svm_cv,
  LogisticRegression = lr_cv
))

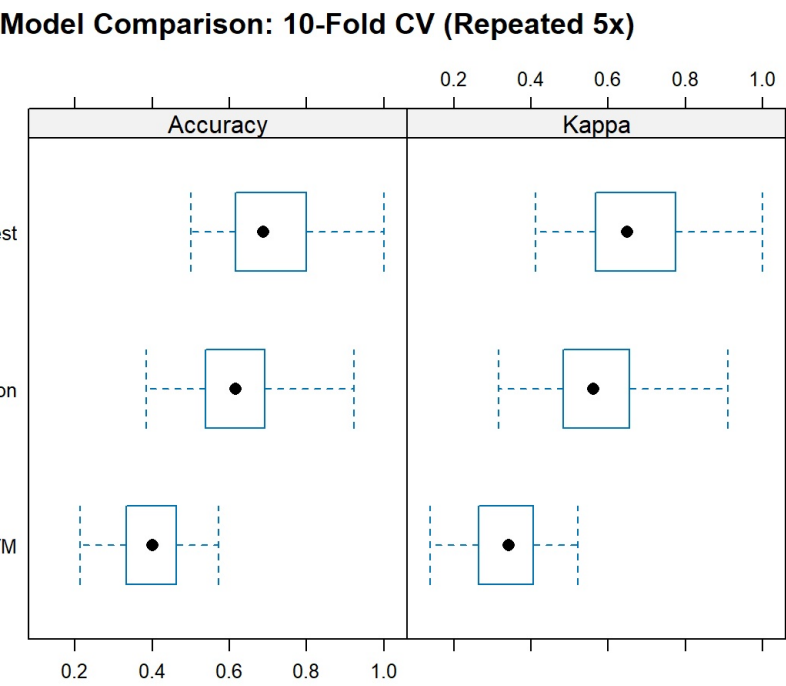
cat("\n=== MODEL COMPARISON (50 CV ITERATIONS EACH) ===\n")
```

```
##
## === MODEL COMPARISON (50 CV ITERATIONS EACH) ===
```

```
summary(results)
```

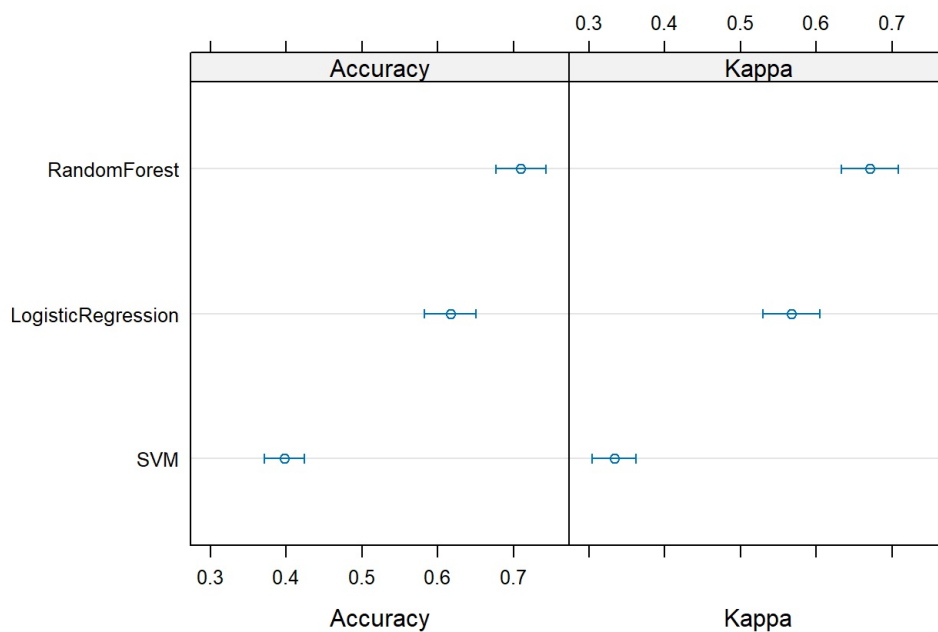
```
##
## Call:
## summary.resamples(object = results)
##
## Models: RandomForest, SVM, LogisticRegression
## Number of resamples: 50
##
## Accuracy
##           Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## RandomForest 0.5000000 0.6177885 0.6875000 0.7100639 0.7964286 1.0000000
## SVM          0.2142857 0.3333333 0.4000000 0.3977224 0.4615385 0.5714286
## LogisticRegression 0.3846154 0.5444712 0.6153846 0.6168022 0.6923077 0.9230769
##           NA's
## RandomForest      0
## SVM                0
## LogisticRegression 0
##
## Kappa
##           Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## RandomForest 0.4117647 0.5692731 0.6488857 0.6710388 0.7700572 1.0000000
## SVM          0.1396648 0.2653899 0.3432162 0.3333090 0.4052288 0.5227273
## LogisticRegression 0.3157895 0.4892348 0.5622846 0.5675973 0.6547653 0.9109589
##           NA's
## RandomForest      0
## SVM                0
## LogisticRegression 0
```

```
#Visualize
bwplot(results, main = "Model Comparison: 10-Fold CV (Repeated 5x)")
```



```
dotplot(results, main = "Model Performance Distributions")
```

Model Performance Distributions



Confidence Level: 0.95

```
#Statistical comparison
cat("\n=== STATISTICAL SIGNIFICANCE TESTS ===\n")
```

```
##
## === STATISTICAL SIGNIFICANCE TESTS ===
```

```
diff_results <- diff(results)
summary(diff_results)
```

```
##
## Call:
## summary.diff.resamples(object = diff_results)
##
## p-value adjustment: bonferroni
## Upper diagonal: estimates of the difference
## Lower diagonal: p-value for H0: difference = 0
##
## Accuracy
##      RandomForest SVM      LogisticRegression
## RandomForest      0.31234  0.09326
## SVM                < 2.2e-16 -0.21908
## LogisticRegression 5.089e-06  4.211e-13
##
## Kappa
##      RandomForest SVM      LogisticRegression
## RandomForest      0.3377  0.1034
## SVM                < 2.2e-16 -0.2343
## LogisticRegression 6.891e-06  8.547e-13
```

```
## Report Final Performance with Confidence Intervals
```

```
#Extract CV results with confidence intervals
```

```
get_cv_stats <- function(model, model_name) {  
  cv_results <- model$resample  
  
  tibble(  
    Model = model_name,  
    Mean_Accuracy = mean(cv_results$Accuracy),  
    SD_Accuracy = sd(cv_results$Accuracy),  
    CI_Lower = mean(cv_results$Accuracy) - 1.96 * sd(cv_results$Accuracy) / sqrt(nrow(cv_results)),  
    CI_Upper = mean(cv_results$Accuracy) + 1.96 * sd(cv_results$Accuracy) / sqrt(nrow(cv_results)),  
    Mean_Kappa = mean(cv_results$Kappa),  
    SD_Kappa = sd(cv_results$Kappa)  
  )  
}
```

```
performance_summary <- bind_rows(  
  get_cv_stats(rf_cv, "Random Forest"),  
  get_cv_stats(svm_cv, "SVM Radial"),  
  get_cv_stats(lr_cv, "Logistic Regression")  
)
```

```
cat("\n=== FINAL MODEL PERFORMANCE (WITH 95% CI) ===\n")
```

```
##  
## === FINAL MODEL PERFORMANCE (WITH 95% CI) ===
```

```
print(performance_summary, digits = 3)
```

```
## # A tibble: 3 × 7
```

##	Model	Mean_Accuracy	SD_Accuracy	CI_Lower	CI_Upper	Mean_Kappa	SD_Kappa
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
## 1	Random Forest	0.710	0.116	0.678	0.742	0.671	0.132
## 2	SVM Radial	0.398	0.0933	0.372	0.424	0.333	0.101
## 3	Logistic Regr...	0.617	0.118	0.584	0.650	0.568	0.132

```
#Select best model
```

```
best_model_name <- performance_summary$Model[which.max(performance_summary$Mean_Accuracy)]  
best_model <- switch(best_model_name,  
  "Random Forest" = rf_cv,  
  "SVM Radial" = svm_cv,  
  "Logistic Regression" = lr_cv)
```

```
cat("\n==> BEST MODEL:", best_model_name, "\n")
```

```
##  
## ==> BEST MODEL: Random Forest
```

```
cat(" Mean CV Accuracy:", round(max(performance_summary$Mean_Accuracy), 3), "\n")
```

```
## Mean CV Accuracy: 0.71
```

```
# Feature Importance Analysis (NO LEAKAGE)
```

```
## Strategy: Bootstrap Stability Selection
```

```
#Bootstrap feature importance across CV folds
```

```
#This ensures features are selected WITHOUT seeing full dataset
```

```
cat("\n=== FEATURE IMPORTANCE: BOOTSTRAP STABILITY APPROACH ===\n")
```

```
##  
## === FEATURE IMPORTANCE: BOOTSTRAP STABILITY APPROACH ===
```

```
cat("Computing importance across 100 bootstrap samples...\n")
```

```
## Computing importance across 100 bootstrap samples...
```

```

set.seed(123)
n_boot <- 100
importance_boot <- matrix(NA, nrow = n_boot, ncol = length(question_cols))
colnames(importance_boot) <- question_cols

for (i in 1:n_boot) {

#Bootstrap sample
boot_idx <- sample(1:nrow(X_full), replace = TRUE)
X_boot <- X_full[boot_idx, ]
y_boot <- y_full[boot_idx]

#Fit RF and extract importance
rf_boot <- randomForest(x = X_boot, y = y_boot, importance = TRUE, ntree = 500)

#Get mean importance across classes
imp_matrix <- importance(rf_boot, type = 1) # Mean decrease in accuracy
importance_boot[i, ] <- rowMeans(as.matrix(imp_matrix))

if (i %% 20 == 0) cat(" Completed", i, "/ 100 bootstrap samples\n")
}

```

```

## Completed 20 / 100 bootstrap samples
## Completed 40 / 100 bootstrap samples
## Completed 60 / 100 bootstrap samples
## Completed 80 / 100 bootstrap samples
## Completed 100 / 100 bootstrap samples

```

```

#Calculate stable importance metrics
importance_stable <- tibble(
  Question = colnames(importance_boot),
  Mean_Importance = colMeans(importance_boot),
  SD_Importance = apply(importance_boot, 2, sd),
  CI_Lower = colMeans(importance_boot) - 1.96 * apply(importance_boot, 2, sd),
  CI_Upper = colMeans(importance_boot) + 1.96 * apply(importance_boot, 2, sd),
  Stability = apply(importance_boot, 2, function(x) sum(x > median(importance_boot)) / n_boot)
) %>%
arrange(desc(Mean_Importance))

cat("\n=== TOP 15 FEATURES: BOOTSTRAP STABLE IMPORTANCE ===\n")

```

```

##
## === TOP 15 FEATURES: BOOTSTRAP STABLE IMPORTANCE ===

```

```

print(importance_stable %>% slice_head(n = 15))

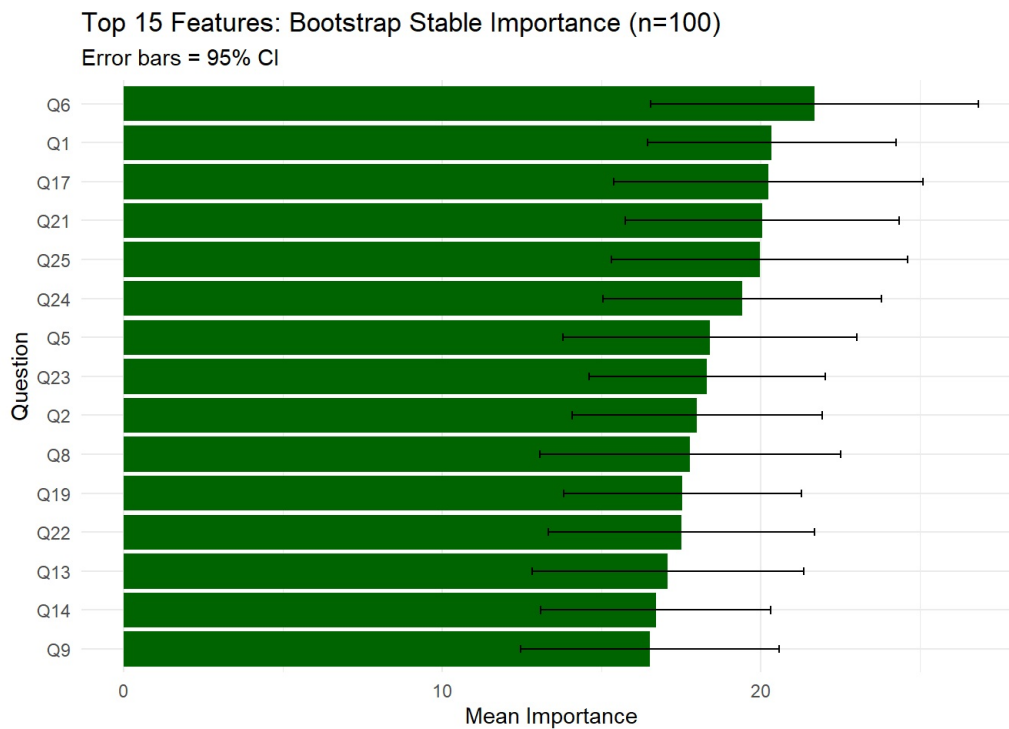
```

```

## # A tibble: 15 × 6
##   Question Mean_Importance SD_Importance CI_Lower CI_Upper Stability
##   <chr>          <dbl>          <dbl>    <dbl>    <dbl>    <dbl>
## 1 Q6             21.7            2.63     16.5     26.8     0.96
## 2 Q1             20.3            1.99     16.4     24.2     0.98
## 3 Q17            20.2            2.48     15.4     25.1     0.95
## 4 Q21            20.1            2.19     15.8     24.4     0.96
## 5 Q25            20.0            2.37     15.3     24.6     0.92
## 6 Q24            19.4            2.23     15.0     23.8     0.9
## 7 Q5             18.4            2.36     13.8     23.0     0.8
## 8 Q23            18.3            1.89     14.6     22.0     0.82
## 9 Q2             18.0            2.00     14.1     21.9     0.74
## 10 Q8            17.8            2.41     13.1     22.5     0.72
## 11 Q19           17.5            1.90     13.8     21.3     0.69
## 12 Q22           17.5            2.13     13.3     21.7     0.67
## 13 Q13           17.1            2.17     12.8     21.3     0.55
## 14 Q14           16.7            1.84     13.1     20.3     0.54
## 15 Q9            16.5            2.07     12.5     20.6     0.46

```

```
ggplot(importance_stable[1:15,], aes(x = reorder(Question, Mean_Importance),
y = Mean_Importance)) +
geom_col(fill = "darkgreen") +
geom_errorbar(aes(ymin = CI_Lower, ymax = CI_Upper), width = 0.2) +
coord_flip() +
theme_minimal() +
labs(title = "Top 15 Features: Bootstrap Stable Importance (n=100)",
subtitle = "Error bars = 95% CI",
x = "Question", y = "Mean Importance")
```



```
## Alternative: Permutation Importance (CV-Based)
```

```
#Compute permutation importance within CV framework
cat("\n=== FEATURE IMPORTANCE: CV-BASED PERMUTATION ===\n")
```

```
##
## === FEATURE IMPORTANCE: CV-BASED PERMUTATION ===
```

```
set.seed(123)
vi_perm_cv <- vip::vi_permute(
  object = best_model,
  feature_names = question_cols,
  train = as.data.frame(lapply(X_full, as.numeric)),
  target = y_full,
  metric = "Accuracy",
  nsim = 50,
  pred_wrapper = function(object, newdata) predict(object, newdata = newdata)
)

vi_perm_cv <- vi_perm_cv %>%
  rename(Variable = Question, Perm_Importance = Importance) %>%
  arrange(desc(Perm_Importance))

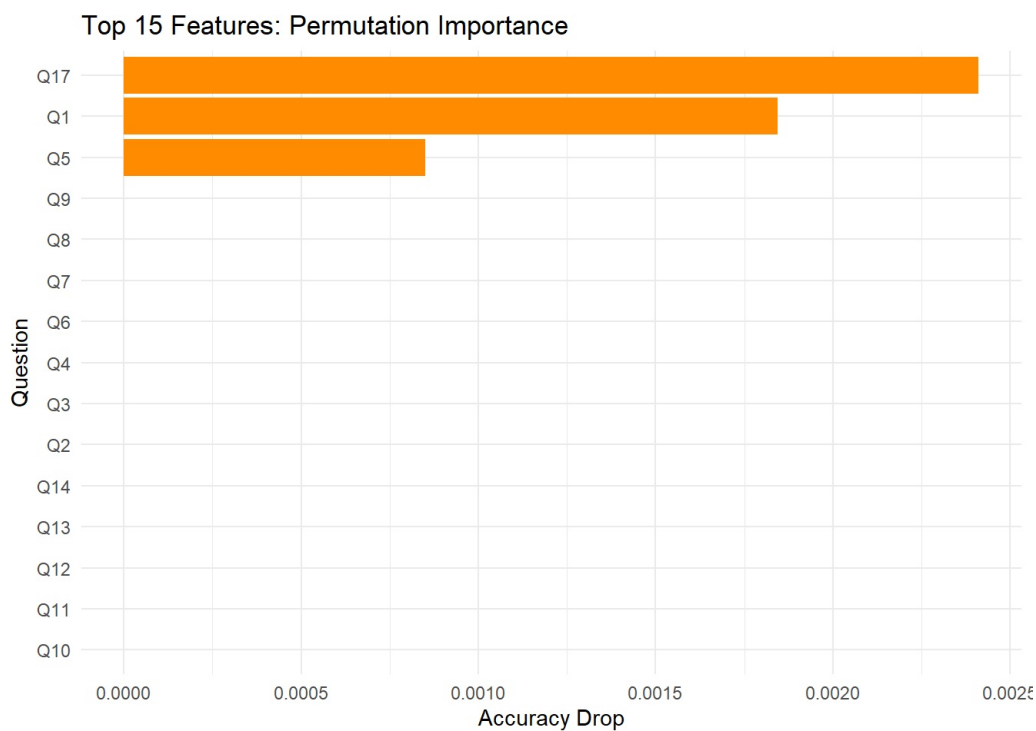
cat("\n=== TOP 15 FEATURES: PERMUTATION IMPORTANCE ===\n")
```

```
##
## === TOP 15 FEATURES: PERMUTATION IMPORTANCE ===
```

```
print(vi_perm_cv %>% slice_head(n = 15))
```

```
## # A tibble: 15 × 3
##   Question Perm_Importance StDev
##   <chr>         <dbl>   <dbl>
## 1 Q17           0.00241 0.00368
## 2 Q1           0.00184 0.00314
## 3 Q5           0.000851 0.00233
## 4 Q2           0         0
## 5 Q3           0         0
## 6 Q4           0         0
## 7 Q6           0         0
## 8 Q7           0         0
## 9 Q8           0         0
## 10 Q9          0         0
## 11 Q10         0         0
## 12 Q11         0         0
## 13 Q12         0         0
## 14 Q13         0         0
## 15 Q14         0         0
```

```
ggplot(vi_perm_cv[1:15,], aes(x = reorder(Question, Perm_Importance),
y = Perm_Importance)) +
  geom_col(fill = "darkorange") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Top 15 Features: Permutation Importance",
x = "Question", y = "Accuracy Drop")
```



```
## SHAP Values (Full Dataset - Descriptive Only)

#SHAP on full model for interpretation (not for selection)
#ACKNOWLEDGE: This uses full dataset, so for INTERPRETATION not SELECTION
cat("\n=== SHAP VALUES: GLOBAL FEATURE IMPORTANCE (INTERPRETIVE) ===\n")
```

```
##
## === SHAP VALUES: GLOBAL FEATURE IMPORTANCE (INTERPRETIVE) ===
```

```
cat("NOTE: SHAP computed on full dataset for interpretation only.\n")
```

```
## NOTE: SHAP computed on full dataset for interpretation only.
```

```
cat("Feature SELECTION uses bootstrap/permutation methods above.\n\n")
```

```
## Feature SELECTION uses bootstrap/permutation methods above.
```

```

rf_final <- best_model$finalModel
X_numeric <- as.data.frame(lapply(X_full, as.numeric))

predict_wrap <- function(object, newdata) {
  as.matrix(predict(object, newdata = newdata, type = "prob"))
}

set.seed(123)
bg_idx <- sample(1:nrow(X_numeric), min(100, nrow(X_numeric)))
bg <- X_numeric[bg_idx, ]

set.seed(123)
shap_values <- fastshap::explain(
  object = rf_final,
  X = X_numeric,
  pred_wrapper = predict_wrap,
  nsim = 100,
  newdata = bg
)

if (is.list(shap_values)) {
  shap_mat <- Reduce("+", lapply(shap_values, abs)) / length(shap_values)
} else {
  shap_mat <- abs(shap_values)
}

shap_summary <- tibble(
  Question = colnames(shap_mat),
  Mean_SHAP = colMeans(shap_mat, na.rm = TRUE)
) %>%
  arrange(desc(Mean_SHAP))

cat("\n=== TOP 15 FEATURES: MEAN |SHAP| ===\n")

```

```

##
## === TOP 15 FEATURES: MEAN |SHAP| ===

```

```

print(shap_summary %>% slice_head(n = 15))

```

```

## # A tibble: 15 × 2
##   Question Mean_SHAP
##   <chr>      <dbl>
## 1 Q23      7.19e-20
## 2 Q5       6.64e-20
## 3 Q2       4.36e-20
## 4 Q7       4.18e-20
## 5 Q26      3.65e-20
## 6 Q4       3.52e-20
## 7 Q18      2.87e-20
## 8 Q21      2.81e-20
## 9 Q1       2.63e-20
## 10 Q25     2.31e-20
## 11 Q9      2.17e-20
## 12 Q15     1.96e-20
## 13 Q3      1.82e-20
## 14 Q13     1.76e-20
## 15 Q20     1.61e-20

```

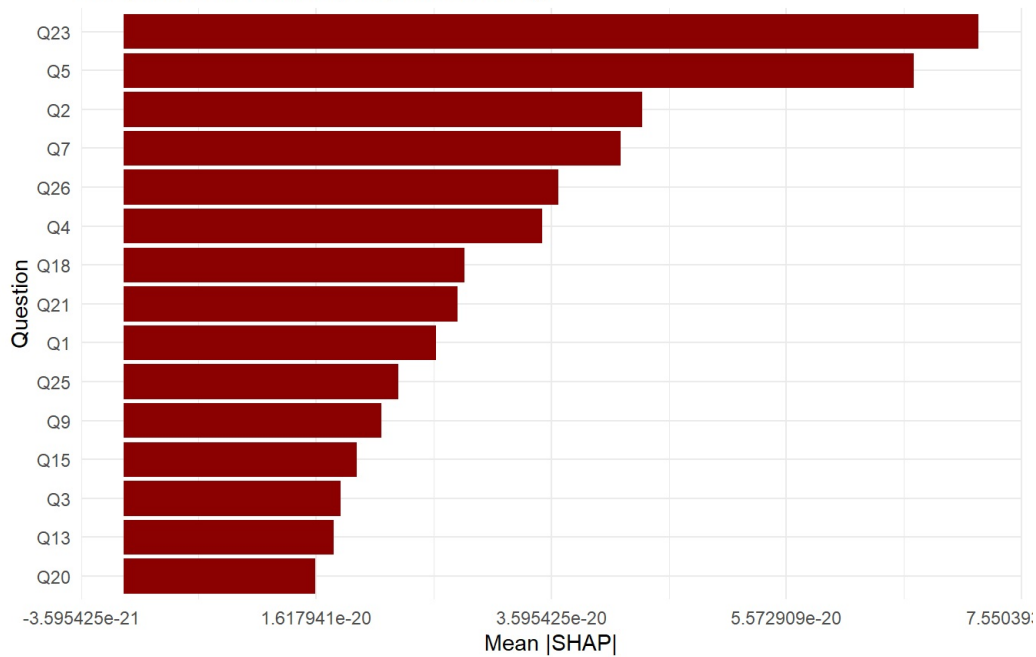
```

ggplot(shap_summary[1:15,], aes(x = reorder(Question, Mean_SHAP), y = Mean_SHAP)) +
  geom_col(fill = "darkred") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Top 15 Features: Mean Absolute SHAP",
  subtitle = "Computed on full dataset (interpretive use only)",
  x = "Question", y = "Mean |SHAP|")

```

Top 15 Features: Mean Absolute SHAP

Computed on full dataset (interpretive use only)



Consensus Ranking (Properly Weighted)

#Combine bootstrap and permutation importance (both CV-based)

#SHAP included but weighted lower since it uses full data

```
consensus_df <- importance_stable %>%
```

```
select(Question, Bootstrap_Imp = Mean_Importance) %>%
```

```
left_join(vi_perm_cv %>% select(Question, Perm_Importance), by = "Question") %>%
```

```
left_join(shap_summary %>% select(Question, SHAP = Mean_SHAP), by = "Question") %>%
```

```
mutate(
```

```
# Normalize to 0-1
```

```
Bootstrap_norm = normalize_01(Bootstrap_Imp),
```

```
Perm_norm = normalize_01(Perm_Importance),
```

```
SHAP_norm = normalize_01(SHAP),
```

```
# Consensus: Bootstrap=50%, Permutation=40%, SHAP=10%
```

```
# (Lower weight for SHAP since it uses full data)
```

```
Consensus_Score = 0.50 * Bootstrap_norm +
```

```
0.40 * Perm_norm +
```

```
0.10 * SHAP_norm
```

```
) %>%
```

```
arrange(desc(Consensus_Score))
```

```
cat("\n=== CONSENSUS FEATURE RANKING (NO LEAKAGE) ===\n")
```

##

```
## === CONSENSUS FEATURE RANKING (NO LEAKAGE) ===
```

```
cat("Weights: Bootstrap=50%, Permutation=40%, SHAP=10%\n\n")
```

```
## Weights: Bootstrap=50%, Permutation=40%, SHAP=10%
```

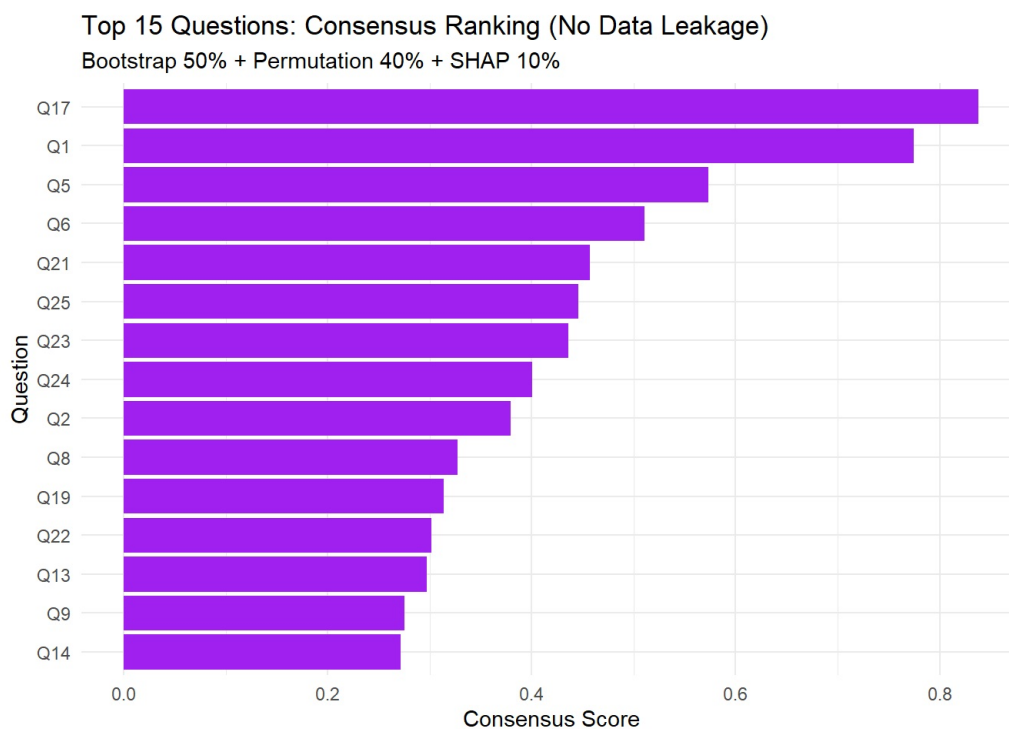
```
print(consensus_df %>%
```

```
select(Question, Consensus_Score, Bootstrap_norm, Perm_norm, SHAP_norm) %>%
```

```
slice_head(n = 15))
```

```
## # A tibble: 15 × 5
##   Question Consensus_Score Bootstrap_norm Perm_norm SHAP_norm
##   <chr>          <dbl>         <dbl>     <dbl>     <dbl>
## 1 Q17            0.838           0.858       1       0.0929
## 2 Q1             0.775           0.869       0.765    0.341
## 3 Q5             0.573           0.679       0.353    0.921
## 4 Q6             0.511           1           0       0.106
## 5 Q21            0.457           0.841       0       0.367
## 6 Q25            0.446           0.832       0       0.295
## 7 Q23            0.436           0.671       0       1
## 8 Q24            0.401           0.779       0       0.112
## 9 Q2             0.379           0.641       0       0.591
## 10 Q8            0.327           0.619       0       0.176
## 11 Q19           0.314           0.595       0       0.162
## 12 Q22           0.302           0.593       0       0.0498
## 13 Q13           0.297           0.551       0       0.216
## 14 Q9            0.275           0.496       0       0.274
## 15 Q14           0.271           0.514       0       0.142
```

```
ggplot(consensus_df[1:15,], aes(x = reorder(Question, Consensus_Score),
y = Consensus_Score)) +
  geom_col(fill = "purple") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Top 15 Questions: Consensus Ranking (No Data Leakage)",
  subtitle = "Bootstrap 50% + Permutation 40% + SHAP 10%",
  x = "Question", y = "Consensus Score")
```



```
#Extract top 10 for survey use
top10_questions <- consensus_df$Question[1:10]
cat("\n=== TOP 10 MOST DISCRIMINATING QUESTIONS (FOR SURVEY) ===\n")
```

```
##
## === TOP 10 MOST DISCRIMINATING QUESTIONS (FOR SURVEY) ===
```

```
cat(paste(top10_questions, collapse = ", "), "\n")
```

```
## Q17, Q1, Q5, Q6, Q21, Q25, Q23, Q24, Q2, Q8
```

```
cat("\nThese questions selected using CV-based methods (no data leakage).\n")
```

```
##
## These questions selected using CV-based methods (no data leakage).
```

```

# Network Analysis: Direction Agreement

#Build network based on direction agreement (sign concordance)
build_direction_network <- function(question_subset, data_df,
threshold = 0.7, title = NULL) {

#Extract answer direction (sign)
A <- sign(as.matrix(select(data_df, all_of(question_subset))))
n_nodes <- nrow(A)

#Calculate pairwise direction agreement
edges <- list()
idx <- 1
for (i in 1:(n_nodes-1)) {
  for (j in (i+1):n_nodes) {
    agreement <- mean(A[i, ] == A[j, ], na.rm = TRUE)
    if (!is.na(agreement) && agreement >= threshold) {
      edges[[idx]] <- data.frame(from = i, to = j, weight = agreement)
      idx <- idx + 1
    }
  }
}

if (length(edges) == 0) {
  message("No edges above threshold for ", title)
  return(NULL)
}

edges_df <- bind_rows(edges)

nodes_df <- tibble(
  id = 1:n_nodes,
  label = data_df$candidate_name,
  party = data_df$party,
  title = paste0(data_df$candidate_name, " (", data_df$party, ")")
)

g <- igraph::graph_from_data_frame(edges_df, vertices = nodes_df, directed = FALSE)

#Assign colors by party
parties <- sort(unique(nodes_df$party))
pal <- viridis::turbo(length(parties))
party_col_map <- setNames(pal, parties)
V(g)$color <- party_col_map[as.character(V(g)$party)]

#Plot network
p <- ggraph(g, layout = "fr") +
  geom_edge_link(aes(width = weight), alpha = 0.5, color = "grey60") +
  geom_node_point(aes(color = party), size = 3) +
  geom_node_text(aes(label = label), repel = TRUE, size = 2.5) +
  scale_color_manual(values = party_col_map) +
  scale_edge_width(range = c(0.5, 2)) +
  theme_void() +
  labs(title = title)

print(p)

return(list(graph = g, nodes = nodes_df, edges = edges_df))
}

#Network based on top 10 consensus questions
cat("\n=== BUILDING NETWORK: TOP 10 CONSENSUS QUESTIONS ===\n")

```

```

##
## === BUILDING NETWORK: TOP 10 CONSENSUS QUESTIONS ===

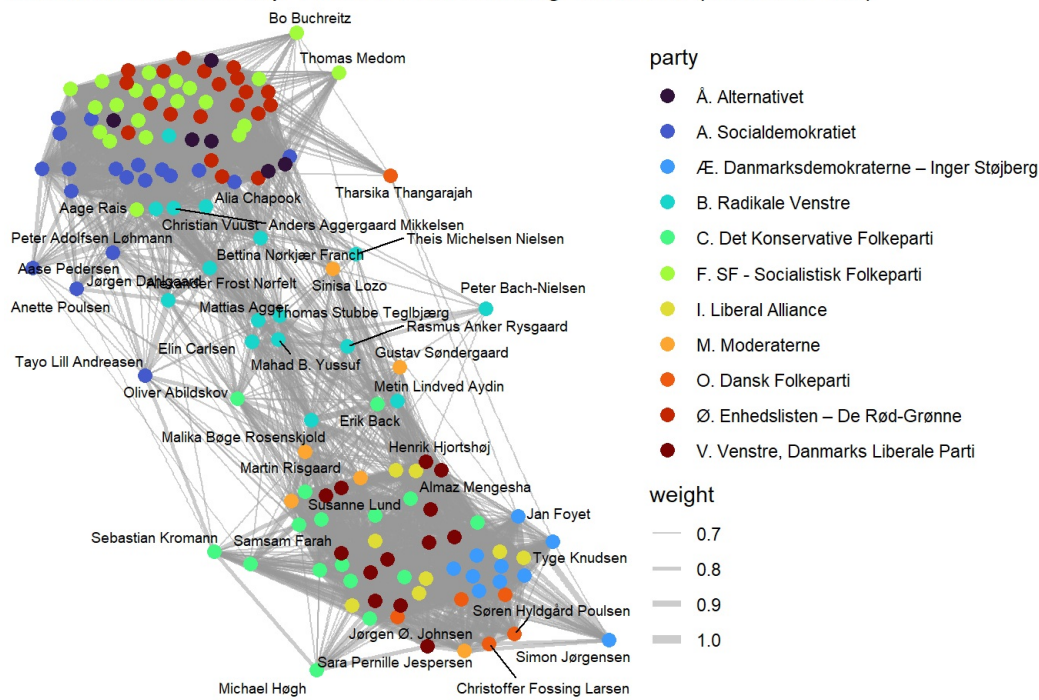
```

```

network_top10 <- build_direction_network(
  question_subset = top10_questions,
  data_df = df,
  threshold = 0.7,
  title = "Candidate Network: Top 10 Most Discriminating Questions (threshold=0.7)"
)

```

Candidate Network: Top 10 Most Discriminating Questions (threshold=0.7)



```
#Network statistics
if (!is.null(network_top10)) {
  g <- network_top10$graph
  cat("\nNetwork Statistics:\n")
  cat(" Nodes:", vcount(g), "\n")
  cat(" Edges:", ecoun(g), "\n")
  cat(" Density:", edge_density(g), "\n")
  cat(" Average degree:", mean(degree(g)), "\n")
}
```

```
##
## Network Statistics:
## Nodes: 141
## Edges: 3538
## Density: 0.35846
## Average degree: 50.1844
```

```
# Session Information
```

```
sessionInfo()
```

```
## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Polish_Poland.utf8 LC_CTYPE=Polish_Poland.utf8
## [3] LC_MONETARY=Polish_Poland.utf8 LC_NUMERIC=C
## [5] LC_TIME=Polish_Poland.utf8
##
## time zone: Europe/Paris
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] cluster_2.1.8.2      stringr_1.6.0      fpc_2.2-14
## [4] vip_0.4.6            ggcorrplot_0.1.4.1  pheatmap_1.0.13
## [7] viridis_0.6.5        viridisLite_0.4.3  visNetwork_2.1.4
## [10] ggraph_2.2.2         tidygraph_1.3.1    igraph_2.3.0
## [13] tibble_3.3.1         fastshap_0.1.1     MASS_7.3-65
## [16] nnet_7.3-20          randomForest_4.7-1.2 caret_7.0-1
## [19] lattice_0.22-9       psych_2.6.3        factoextra_2.0.0
## [22] ggplot2_4.0.3        tidyr_1.3.2        dplyr_1.2.1
## [25] readr_2.2.0
##
## loaded via a namespace (and not attached):
## [1] RColorBrewer_1.1-3   rstudioapi_0.18.0   jsonlite_2.0.0
## [4] magrittr_2.0.5       modeltools_0.2-24   farver_2.1.2
## [7] rmarkdown_2.31       vctrs_0.7.3         memoise_2.0.1
## [10] rstatix_0.7.3        htmltools_0.5.9     broom_1.0.12
## [13] Formula_1.2-5        pROC_1.19.0.1       sass_0.4.10
## [16] parallelly_1.47.0    bslib_0.10.0        htmlwidgets_1.6.4
## [19] plyr_1.8.9           lubridate_1.9.5     cachem_1.1.0
## [22] lifecycle_1.0.5      iterators_1.0.14    pkgconfig_2.0.3
## [25] Matrix_1.7-5         R6_2.6.1            fastmap_1.2.0
## [28] future_1.70.0        digest_0.6.39       ggpubr_0.6.3
## [31] labeling_0.4.3       yardstick_1.4.0     timechange_0.4.0
## [34] polyclip_1.10-7      abind_1.4-8         compiler_4.6.0
## [37] proxy_0.4-29         bit64_4.8.0         withr_3.0.2
## [40] S7_0.2.2            backports_1.5.1     carData_3.0-6
## [43] ggforce_0.5.0        ggsignif_0.6.4      lava_1.9.0
## [46] ModelMetrics_1.2.2.2 tools_4.6.0          otel_0.2.0
## [49] prabclus_2.3-5       future.apply_1.20.2 glue_1.8.1
## [52] nlme_3.1-169         grid_4.6.0          reshape2_1.4.5
## [55] generics_0.1.4       recipes_1.3.2       gtable_0.3.6
## [58] tzdb_0.5.0          class_7.3-23        data.table_1.18.2.1
## [61] hms_1.1.4           car_3.1-5           utf8_1.2.6
## [64] flexmix_2.3-20       ggrepel_0.9.8       foreach_1.5.2
## [67] pillar_1.11.1        vroom_1.7.1         robustbase_0.99-7
## [70] splines_4.6.0        tweenr_2.0.3        survival_3.8-6
## [73] bit_4.6.0           tidyselect_1.2.1    knitr_1.51
## [76] gridExtra_2.3        stats4_4.6.0        xfun_0.57
## [79] graphlayouts_1.2.3   hardhat_1.4.3       diptest_0.77-2
## [82] timeDate_4052.112    DEoptimR_1.1-4      stringi_1.8.7
## [85] yaml_2.3.12         evaluate_1.0.5      codetools_0.2-20
## [88] kernlab_0.9-33       cli_3.6.6           rpart_4.1.27
## [91] jquerylib_0.1.4      Rcpp_1.1.1-1.1     globals_0.19.1
## [94] parallel_4.6.0       gower_1.0.2         mclust_6.1.2
## [97] listenv_0.10.1       ipred_0.9-15        scales_1.4.0
## [100] prodlim_2026.03.11  e1071_1.7-17        purrr_1.2.2
## [103] crayon_1.5.3         rlang_1.2.0         mnormt_2.1.2
```

```
# This gives you everything at once
cat("Column names:\n")
```

```
## Column names:
```

```
print(names(df))
```

```
## [1] "candidate_name" "party"      "Q1"      "Q2"
## [5] "Q3"              "Q4"      "Q5"      "Q6"
## [9] "Q7"              "Q8"      "Q9"      "Q10"
## [13] "Q11"             "Q12"     "Q13"     "Q14"
## [17] "Q15"             "Q16"     "Q17"     "Q18"
## [21] "Q19"             "Q20"     "Q21"     "Q22"
## [25] "Q23"             "Q24"     "Q25"     "Q26"
```

```
cat("\nParty distribution:\n")
```

```
##
## Party distribution:
```

```
print(table(df$party))
```

```
##
##                Å. Alternativet
##                        6
##                A. Socialdemokratiet
##                        19
##  Æ. Danmarksdemokraterne – Inger Støjberg
##                        10
##                B. Radikale Venstre
##                        16
##                C. Det Konservative Folkeparti
##                        16
##                F. SF - Socialistisk Folkeparti
##                        21
##                I. Liberal Alliance
##                        8
##                M. Moderaterne
##                        6
##                O. Dansk Folkeparti
##                        6
##                Ø. Enhedslisten – De Rød-Grønne
##                        20
##                V. Venstre, Danmarks Liberale Parti
##                        13
```

```
cat("\nNumber of unique parties:", length(unique(df$party)), "\n")
```

```
##
## Number of unique parties: 11
```